Table of Contents

1. Task 1 - Dataset Selection 4
   1.1 Introduction to the Dataset 4
   1.2 Justification for Selection 4
   1.3 Dataset Attribute Description 6
   1.4 ER Diagram 6
2. Task 2 - Preparation of Data Sources 9
   2.1 Overview of Data Sources 9
   2.2 Source 1 - Booking Data (booking_data.csv) 9
   2.3 Source 2 - Guest Data (guest_data.csv) 9
   2.4 Source 3 – Reservation Status Data (reservation_status_data.xlsx) 9
   2.5 Data Source Summary 9
3. Task 3 - Solution Architecture 10
   3.1 Solution Architecture Overview 10
   3.2 Source Layer 10
   3.3 Staging Layer 11
   3.4 SSIS ETL Engine 11
   3.5 Data Warehouse Layer 12
   3.6 Reporting Layer 12
4. Task 4 - Data Warehouse Design and Development 13
   4.1 Dimensional Model Overview 13
   4.2 Grain Definition 13
   4.3 Fact Table - FactBooking 13
   4.4 Star Schema Diagram 13
   4.5 Staging Tables 15
   4.6 Dimension Tables 15
   4.7 Slowly Changing Dimension 16
   4.8 Assumptions 16
5. Task 5 - ETL Development 17
   5.1 ETL Process Overview 17
   5.2 Data Profiling Task 19
   5.3 Staging Layer (staging.dtsx) 21
   5.4 Validation(Staging tables loaded to SSMS): 24

5.4 Dimension Load Layer (Dimensions.dtsx) 25
5.4.1 DimHotel 26
5.4.2 DimRoom 27
5.4.3 DimAgent 29
..... 32
5.4.4 DimGuest (SCD Type 2 Implementation) 32
5.4.5 DimDate 37
5.5 Load FactBooking (Fact.dtsx) 39
6. Task 6 - ETL Development - Accumulating fact tables 44
6.1 Accumulating Fact Table Overview 44
6.2 Step 1 - Extending the Fact Table 44
6.3 Step 2 - Setting accm_txn_create_time During Initial Load 45
6.4 Step 3 - Preparing the Completion Dataset 47
6.5 Step 4 - Update_AccumulatingFact.dtsx Package 47
6.6 Step 5 – Processing Time Calculation 51
6.7 Validation 51
6.8 Summary 51

1. Task 1 - Dataset Selection

1.1 Introduction to the Dataset

The dataset selected for this project is a **Hotel Booking Management System** dataset, which contains detailed records of hotel reservations, guest information, and booking outcomes. The dataset originates from a real-world hotel environment, and all personally identifiable information such as names, emails, phone numbers, and credit card details has been synthetically generated to ensure privacy.

To simulate a realistic heterogeneous data environment, the data is distributed across multiple data source formats, including both CSV and Excel files:

1. **booking_data.csv (CSV):** Contains core booking and transactional details such as hotel type, lead time, arrival date components, length of stay, guest composition, meal preferences, market segment, distribution channel, room allocation details, deposit type, and customer classification.
2. **guest_data.csv (CSV):** Contains guest-related information including name, email, phone number, credit card details (synthetic), and country of origin.
3. **reservation_status_data.xlsx (xlsx):** Contains booking outcome and performance-related data such as cancellation status, Average Daily Rate (ADR), required car parking spaces, total special requests, reservation status, and reservation status date.

1.2 Justification for Selection

The Hotel Booking dataset was selected because it satisfies all the requirements outlined in the assignment for a suitable OLTP dataset.

Each record in the dataset represents a **single transactional booking event**, rather than pre-aggregated or summarized data, making it a true OLTP source suitable for ETL processing and dimensional modeling. The dataset consists of over **119,000 records**, providing sufficient volume to populate fact and dimension tables, build OLAP cubes, and generate meaningful analytical reports.

The dataset includes a rich set of attributes distributed across the three sources, covering multiple business perspectives such as bookings, guests, and reservation outcomes. This enables the design of a **comprehensive dimensional model**, including dimensions such as

Guest, Hotel, Room, Agent, and Date, along with fact tables capturing booking transactions and performance metrics.

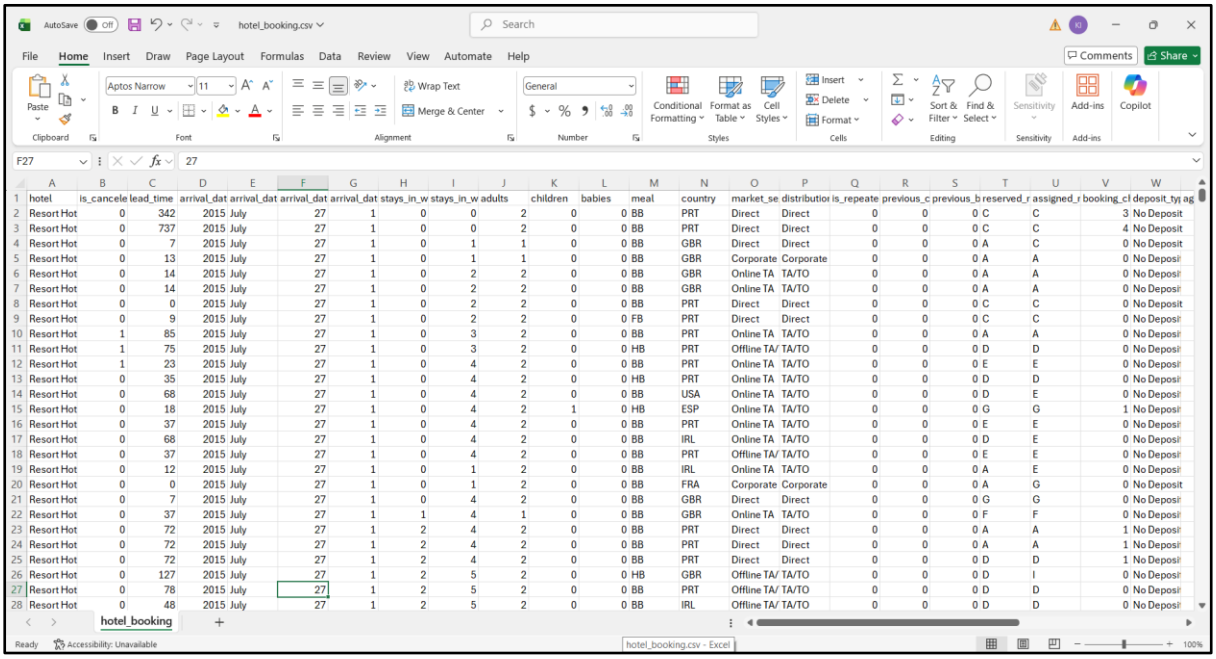
Additionally, the dataset supports **natural hierarchies**, such as:

- Date hierarchy (day → month → year)
- Geographic hierarchy (country → region/continent)
- Room-related categorization (room type classifications)

Certain attributes, particularly within the guest domain (e.g., country or customer type), may evolve over time, making them suitable candidates for implementing **Slowly Changing Dimensions (SCD Type 2)**.

Furthermore, the hotel booking domain provides a **unique and realistic business scenario**, distinct from commonly used academic datasets such as retail or sales systems, thereby enhancing the practical relevance of this project.

Dataset Link: https://www.kaggle.com/datasets/saadharoon27/hotel-booking-dataset



hotel	is_cancele	lead_time	arrival_dat	arrival_dat	arrival_dat	arrival_dat	stays_in_w	stays_in_w	adults	children	babies	meal	country	market_se	distributio	is_repeate	previous_c	previous_b	reserved_r	assigned_r	booking_cl	deposit_ty	ag
Resort Hot	0	342	2015	July	27	1	0	0	2	0	0	BB	PRT	Direct	Direct	0	0	0	C	C	3	No Deposit	
Resort Hot	0	737	2015	July	27	1	0	0	2	0	0	BB	PRT	Direct	Direct	0	0	0	C	C	4	No Deposit	
Resort Hot	0	7	2015	July	27	1	0	1	1	0	0	BB	GBR	Direct	Direct	0	0	0	A	C	0	No Deposit	
Resort Hot	0	13	2015	July	27	1	0	1	1	0	0	BB	GBR	Corporate	Corporate	0	0	0	A	A	0	No Deposi	
Resort Hot	0	14	2015	July	27	1	0	2	2	0	0	BB	GBR	Online TA	TA/TO	0	0	0	A	A	0	No Deposi	
Resort Hot	0	14	2015	July	27	1	0	2	2	0	0	BB	GBR	Online TA	TA/TO	0	0	0	A	A	0	No Deposi	
Resort Hot	0	0	2015	July	27	1	0	2	2	0	0	BB	PRT	Direct	Direct	0	0	0	C	C	0	No Deposit	
Resort Hot	0	9	2015	July	27	1	0	2	2	0	0	FB	PRT	Direct	Direct	0	0	0	C	C	0	No Deposi	
Resort Hot	1	85	2015	July	27	1	0	3	2	0	0	BB	PRT	Online TA	TA/TO	0	0	0	A	A	0	No Deposi	
Resort Hot	1	75	2015	July	27	1	0	3	2	0	0	HB	PRT	Offline TA/	TA/TO	0	0	0	D	D	0	No Deposi	
Resort Hot	1	23	2015	July	27	1	0	4	2	0	0	BB	PRT	Online TA	TA/TO	0	0	0	E	E	0	No Deposi	
Resort Hot	0	35	2015	July	27	1	0	4	2	0	0	HB	PRT	Online TA	TA/TO	0	0	0	D	D	0	No Deposi	
Resort Hot	0	68	2015	July	27	1	0	4	2	0	0	BB	USA	Online TA	TA/TO	0	0	0	D	E	0	No Deposi	
Resort Hot	0	18	2015	July	27	1	0	4	2	1	0	HB	ESP	Online TA	TA/TO	0	0	0	G	G	1	No Deposi	
Resort Hot	0	37	2015	July	27	1	0	4	2	0	0	BB	PRT	Online TA	TA/TO	0	0	0	E	E	0	No Deposi	
Resort Hot	0	68	2015	July	27	1	0	4	2	0	0	BB	IRL	Online TA	TA/TO	0	0	0	D	E	0	No Deposi	
Resort Hot	0	37	2015	July	27	1	0	4	2	0	0	BB	PRT	Offline TA/	TA/TO	0	0	0	E	E	0	No Deposi	
Resort Hot	0	12	2015	July	27	1	0	1	2	0	0	BB	IRL	Online TA	TA/TO	0	0	0	A	E	0	No Deposi	
Resort Hot	0	0	2015	July	27	1	0	1	2	0	0	BB	FRA	Corporate	Corporate	0	0	0	A	G	0	No Deposit	
Resort Hot	0	7	2015	July	27	1	0	4	2	0	0	BB	GBR	Direct	Direct	0	0	0	G	G	0	No Deposi	
Resort Hot	0	37	2015	July	27	1	1	4	1	0	0	BB	GBR	Online TA	TA/TO	0	0	0	F	F	0	No Deposi	
Resort Hot	0	72	2015	July	27	1	2	4	2	0	0	BB	PRT	Direct	Direct	0	0	0	A	A	1	No Deposi	
Resort Hot	0	72	2015	July	27	1	2	4	2	0	0	BB	PRT	Direct	Direct	0	0	0	A	A	1	No Deposi	
Resort Hot	0	72	2015	July	27	1	2	4	2	0	0	BB	PRT	Direct	Direct	0	0	0	D	D	1	No Deposi	
Resort Hot	0	127	2015	July	27	1	2	5	2	0	0	HB	GBR	Offline TA/	TA/TO	0	0	0	D	I	0	No Deposi	
Resort Hot	0	78	2015	July	27	1	2	5	2	0	0	BB	PRT	Offline TA/	TA/TO	0	0	0	D	D	0	No Deposi	
Resort Hot	0	48	2015	July	27	1	2	5	2	0	0	BB	IRL	Offline TA/	TA/TO	0	0	0	D	D	0	No Deposi	

Figure 1 - Sample of the selected OLTP Dataset

1.3 Dataset Attribute Description

The dataset used in this project contains multiple attributes that represent different aspects of hotel booking transactions. For the purpose of this implementation, these attributes have been logically divided into three separate data sources to better simulate a real-world environment.

The **booking_data.csv** file includes the main booking-related information. This covers details such as the hotel type, lead time (which is the number of days between booking and arrival), and arrival date information including year, month, week number, and day. The length of stay is recorded using stays_in_weekend_nights and stays_in_week_nights. Information about the guests, such as the number of adults, children, and babies, is also included. In addition, this file contains attributes like meal type, market segment, distribution channel, reserved and assigned room types, deposit type, and customer type, which together describe the booking preferences and operational details.

The **guest_data.csv** file focuses on customer-related information. It includes details such as the guest's name, email, phone number, and credit card information, which have been artificially generated. It also contains the country of origin, which can be used for geographic analysis of customers.

The **reservation_status_data.xlsx** file contains information about the outcome of each booking. This includes whether the booking was canceled, as well as the Average Daily Rate (ADR), which represents the revenue per room. Other details such as required car parking spaces and the number of special requests made by guests are also included. The reservation status shows the final state of the booking (such as Check-Out or Canceled), and the reservation status date indicates when that status was last updated.

Overall, dividing the dataset into these three sources helps to better represent how data is stored in real systems. It also makes it easier to perform data integration, design ETL processes, and build a structured data warehouse for analysis.

1.4 ER Diagram

The following Entity-Relationship diagram illustrates the logical structure of the OLTP source data after normalization into related entities. The central entity is the Booking table, which references supporting entities, namely Guest, Hotel, Room, Date and Agent.

It should be noted that the ER diagram represents the logical design of the OLTP data model. In the original dataset, data was available in a denormalized form and distributed across multiple source files in different formats (CSV and Excel). As part of the data preparation stage, this data was logically separated and normalized into distinct entities before being loaded into the data warehouse.

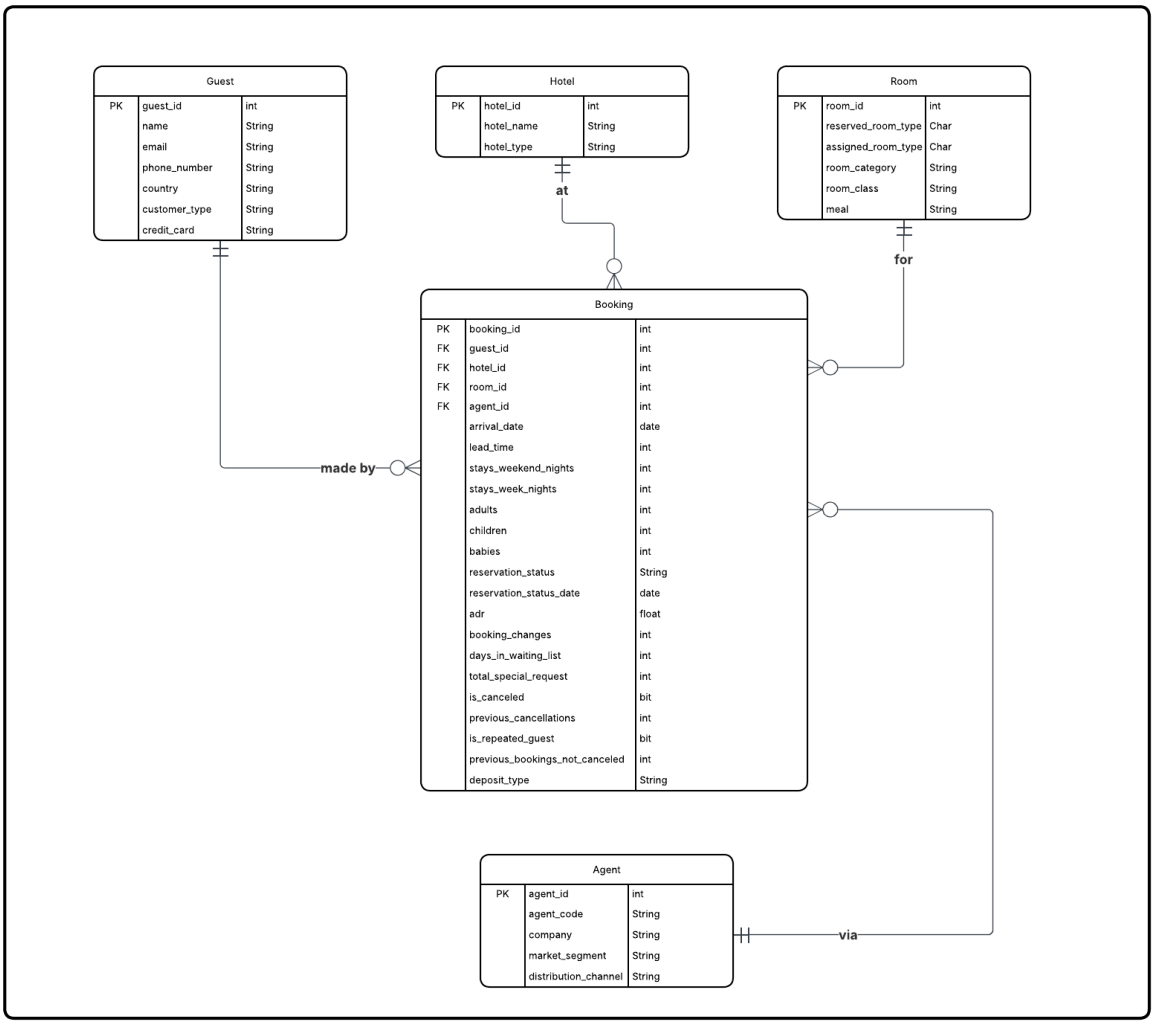


Figure 2 - ER Diagram

The main entities derived from the dataset are described below.

Booking Entity

The Booking entity is the primary transactional entity in the dataset, representing a single hotel reservation event. It contains the core measurable facts such as the Average Daily Rate, length of stay, number of guests, booking changes and special requests. Each booking record is linked to a Guest, Hotel, Room and Agent through foreign key relationships, and this entity forms the basis of the Fact Table in the data warehouse.

Guest Entity

The Guest entity contains personal and demographic information about the customer who made the booking. Key attributes include the name, email address, phone number, country of

origin and customer type of guest. Guest records are stored in a separate CSV file as one of the multiple data source types used in this assignment. The country and customer type attributes are identified as slowly changing attributes, which makes the Guest dimension a suitable candidate for Slowly Changing Dimension Type 2 implementation in the data warehouse.

Hotel Entity

The Hotel entity captures the type and name of the hotel where the booking was made. The dataset includes two hotel types, which are the Resort Hotel and the City Hotel. This entity serves as a simple reference dimension in the data warehouse.

Room Entity

The Room entity captures room-related information including the reserved room type, which is the room code originally booked, and the assigned room type, which is the room provided at check-in. Additional enriched attributes such as room category and room class are introduced manually through an Excel reference file to support hierarchical analysis. The meal plan associated with the booking is also stored within this entity.

Agent Entity

The Agent entity captures the booking channel and agent details, including the agent identifier, market segment and distribution channel. This entity is derived from booking data and represents booking channel information such as market segment and distribution channel.

Date Entity

The Date entity represents the temporal aspects of each booking. It is derived from attributes such as arrival date and reservation status date, which are originally stored in separate columns within the dataset. These attributes are transformed into a structured date format during the data preparation stage.

2. Task 2 - Preparation of Data Sources

2.1 Overview of Data Sources

The dataset used in this project has been organized into three separate data sources to support a realistic ETL process and simulate a heterogeneous data environment. These sources are structured to represent different aspects of hotel operations, including bookings, guest information, and reservation outcomes.

2.2 Source 1 - Booking Data (booking_data.csv)

The core transactional booking information is stored in a CSV file named **booking_data.csv**, which contains a large number of booking records. This file includes key attributes such as hotel type, lead time, arrival date details, length of stay, guest composition, meal preferences, market segment, distribution channel, room reservation and assignment details, deposit type, and customer classification.

This dataset serves as the primary source for constructing the **FactBooking table**, and also contributes to the **DimHotel, DimRoom, DimAgent, and DimDate dimensions** during the ETL process.

2.3 Source 2 - Guest Data (guest_data.csv)

Guest-related information is stored in a CSV file named **guest_data.csv**. This file contains attributes such as guest name, email address, phone number, credit card details (synthetically generated), and country of origin.

This source is used to build the **DimGuest dimension**, which enables analysis of customer demographics and geographic distribution.

2.4 Source 3 – Reservation Status Data (reservation_status_data.xlsx)

The booking outcome and performance-related information is stored in a CSV file named **reservation_status_data.xlsx**. It includes attributes such as cancellation status, Average Daily Rate (ADR), required car parking spaces, total number of special requests, reservation status, and reservation status date.

This source contributes mainly to the **FactBooking table**, particularly for analyzing booking performance, revenue trends, and cancellation behavior.

2.5 Data Source Summary

The following table summarizes the information available in each source and its intended use in the data warehouse.

Data Type	Source File	Format	Used for
Booking transactions	booking_data.csv	CSV	FactBooking, DimHotel, DimRoom, DimAgent, DimDate
Guest information	guest_data.csv	CSV	DimGuest
Reservation outcomes	reservation_status_data.xlsx	Xlsx	FactBooking

3. Task 3 - Solution Architecture

3.1 Solution Architecture Overview

The proposed Data Warehousing and Business Intelligence solution for the Hotel Booking Demand Dataset follows a layered architecture consisting of four main layers, namely the Source Layer, the Staging Layer, the Data Warehouse Layer and the Reporting Layer. Each layer serves a distinct purpose in the overall data flow from source systems to end user reporting.

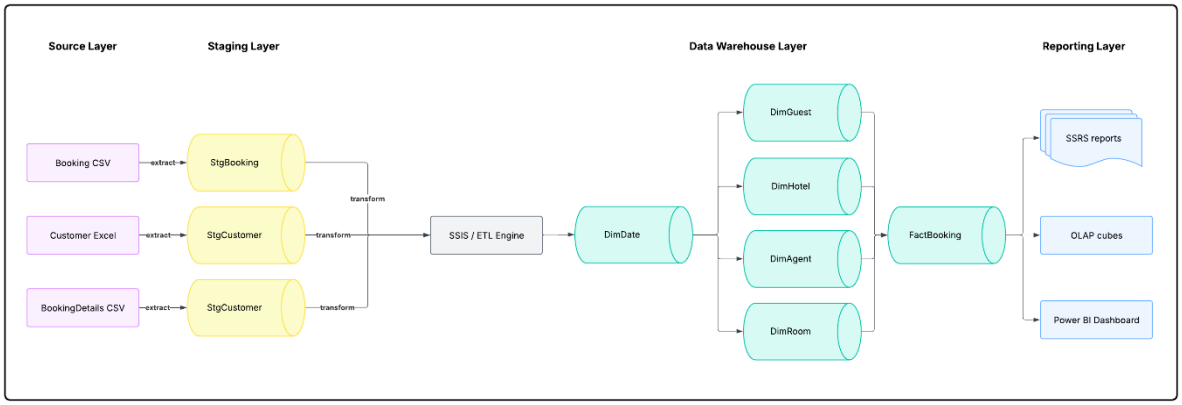


Figure 3 - Solution Architecture

3.2 Source Layer

The Source Layer contains the raw data extracted from multiple heterogeneous data sources in different file formats, including both CSV and Excel files:

- **booking_data.csv** - Contains transactional booking information such as hotel type, arrival dates, stay duration, room details, and booking channels.
- **guest_data.csv** - Contains guest demographic and personal information such as name, country, email, and contact details.
- **reservation_status_data.xlsx** - Contains booking outcomes including cancellation status, ADR, special requests, and reservation status details.

These files represent heterogeneous data sources that simulate real-world operational systems.

3.3 Staging Layer

The Staging Layer acts as a temporary storage area where raw data from source systems is loaded before any transformation is applied. This layer isolates source data from the data warehouse and provides a controlled environment for data cleaning, validation, and preparation.

In this project, three staging tables are used:

- **stg_Booking** - Stores raw transactional booking data extracted from *booking_data.csv*
- **stg_Guest** - Stores guest-related information extracted from *guest_data.csv*
- **stg_ReservationStatus** - Stores booking outcome data extracted from *reservation_status_data.xlsx*

This structure ensures that each source is independently staged, allowing better data control and simplifying the ETL transformation process in SSIS.

3.4 SSIS ETL Engine

The SSIS ETL Engine is responsible for extracting data from the staging tables, applying the necessary transformations and loading the processed data into the Data Warehouse Layer. Transformations performed at this stage include data type conversions, derived column calculations, lookup joints to resolve surrogate keys, Slowly Changing Dimension handling for the Guest dimension and date part assembly for the Date dimension. The SSIS packages are developed using SQL Server Integration Services and are executed in a defined order to ensure that dimension tables are loaded before the fact table.

3.5 Data Warehouse Layer

The Data Warehouse Layer is implemented using a **Star Schema design** in SQL Server. It consists of:

Dimension Tables:

- DimGuest
- DimHotel
- DimRoom
- DimAgent
- DimDate

Fact Table:

- FactBooking

The FactBooking table stores all measurable booking transactions, while dimension tables store descriptive attributes used for filtering, grouping, and analysis.

This structure supports efficient querying and OLAP cube processing in SSAS.

3.6 Reporting Layer

The Reporting Layer provides tools for data analysis and business intelligence reporting, enabling end users to gain meaningful insights from the data warehouse.

This layer includes the following components:

- **SQL Server Reporting Services (SSRS):** Used to generate structured operational and analytical reports based on the data warehouse.
- **SQL Server Analysis Services (SSAS):** Used to build OLAP cubes that support multidimensional analysis across different dimensions and hierarchies.
- **Power BI Dashboards:** Used to create interactive visualizations for analyzing booking trends, revenue performance, and customer behavior.

Overall, this layer supports effective decision-making by transforming warehouse data into actionable business insights.

4. Task 4 - Data Warehouse Design and Development

4.1 Dimensional Model Overview

The data warehouse for the Hotel Booking Demand Dataset is designed using a Star Schema. In this design, a central fact table named FactBooking is surrounded by five-dimension tables, namely DimGuest, DimHotel, DimRoom, DimAgent and DimDate. The Star Schema was selected over the Snowflake Schema because it provides simpler queries, better query performance and easier implementation within SQL Server Analysis Services for OLAP cube creation.

4.2 Grain Definition

The grain of the fact table is defined as one row per hotel booking transaction. This means each record in the FactBooking table represents a single booking made by a guest at a hotel, and all measures in the fact table are recorded at this level of detail.

4.3 Fact Table - FactBooking

The FactBooking table is the central table of the dimensional model and contains the measurable facts related to each hotel booking transaction. It holds foreign keys referencing all five dimension tables and contains the following measures: lead_time which represents the number of days between booking and arrival, stays_weekend_nights and stays_week_nights which capture the length of stay, total_stays and total_guests which are derived measures calculated during the ETL process, adr which represents the Average Daily Rate, total_revenue which is a derived measure calculated by multiplying adr by total_stays, booking_changes, days_in_waiting_list and total_special_requests. The table also contains the is_canceled flag and the reservation_status attribute. Additionally, three columns related to the accumulating fact table pattern are included, namely accm_txn_create_time, accm_txn_complete_time and txn_process_time_hours, which are used in Task 6.

4.4 Star Schema Diagram

The following diagram illustrates the Star Schema dimensional model designed for the Hotel Booking Demand Dataset. The FactBooking table is positioned at the centre of the schema,

surrounded by five-dimensional tables. Foreign key relationships connect each dimension table to the fact table through their respective surrogate keys.

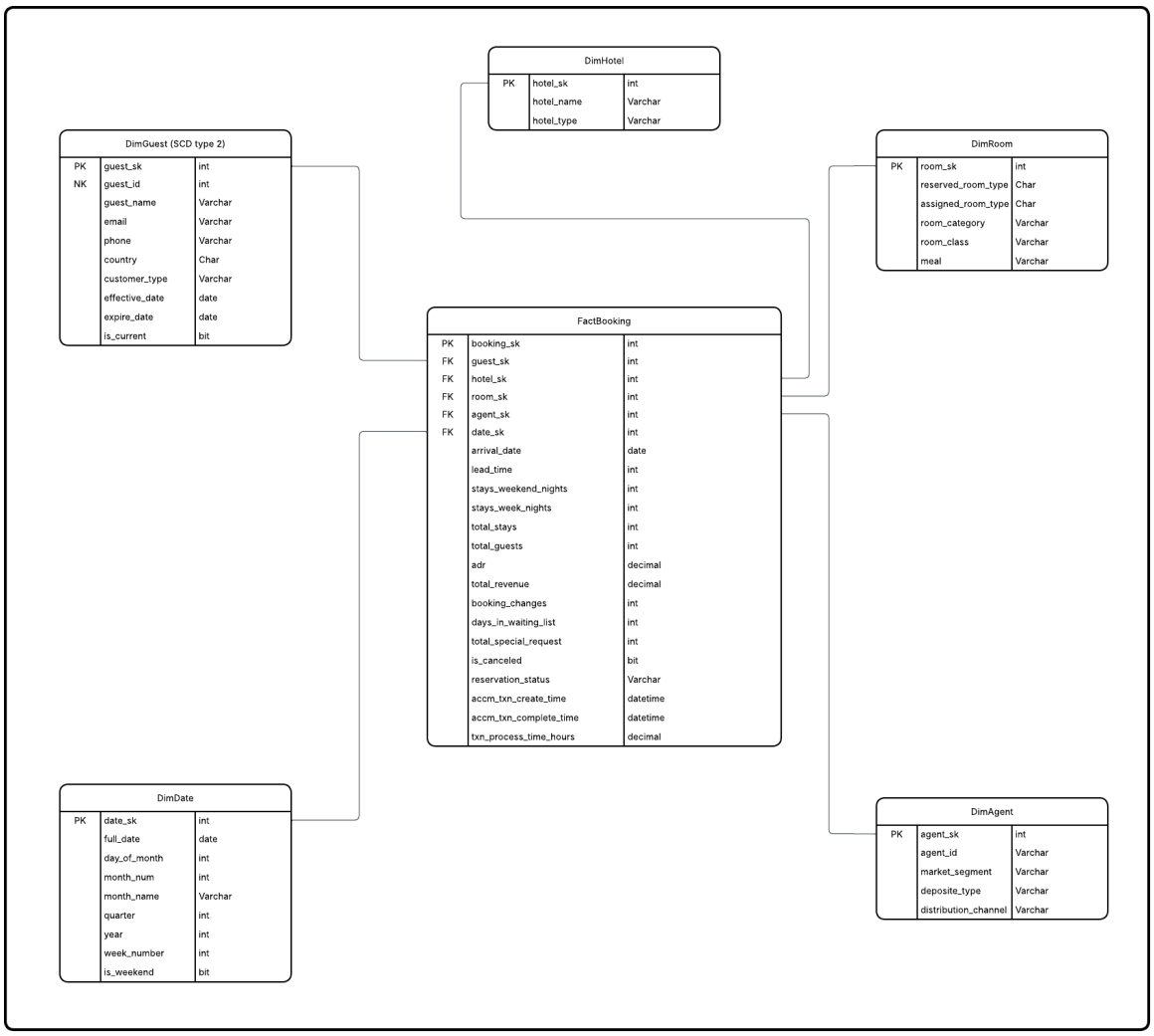
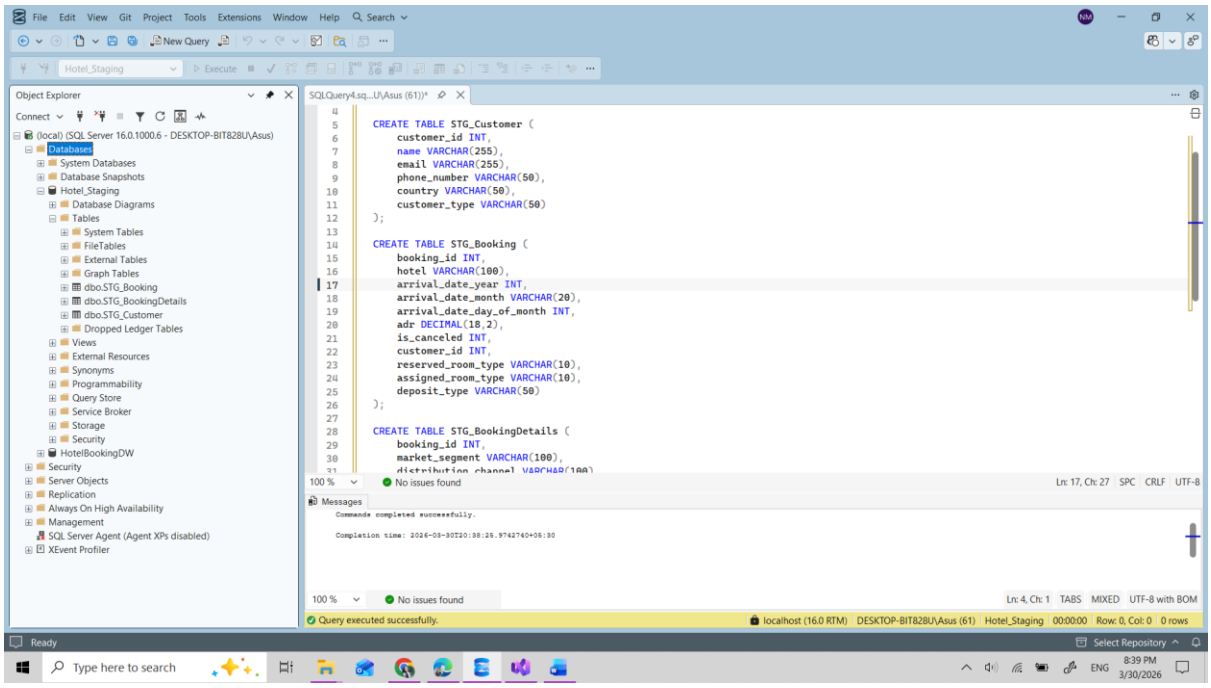


Figure 4 - Star Schema

4.5 Staging Tables



4.6 Dimension Tables

DimGuest stores personal and demographic information about the guest who made the booking. This dimension is implemented as a Slowly Changing Dimension Type 2, meaning that when key attributes such as country or customer type change over time, a new record is inserted rather than overwriting the existing record. The effective_date, expiry_date and is_current columns are used to track the history of changes. The guest_id column serves as the natural key while guest_sk is the surrogate key.

DimHotel is a simple reference dimension that stores the hotel name and hotel type. Since the dataset contains only two hotel types, namely Resort Hotel and City Hotel, this dimension will contain only two records.

DimRoom stores room related attributes including the reserved room type and the assigned room type. This dimension also contains enriched attributes such as room_category and room_class which support hierarchical analysis. The room hierarchy goes from reserved room type up to room category and then to room class.

DimAgent stores booking channel related information including the agent identifier, market segment, distribution channel and deposit type. This dimension supports the analysis of booking patterns across different booking channels and market segments.

DimDate stores date related attributes derived from the arrival date columns in the source data. It supports date hierarchies from day level up to year level, including day of month, month number, month name, quarter, year and week number. A flag named is_weekend identifies whether the date falls on a weekend.

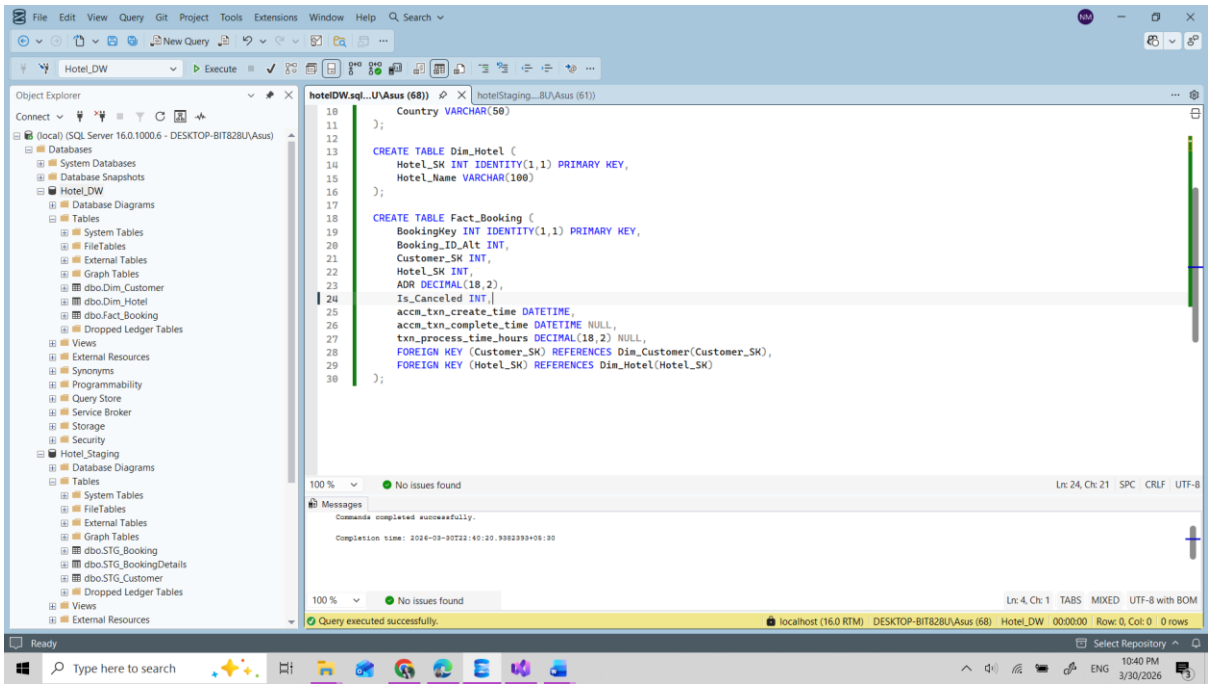


Figure 5 - Query to create dimension tables

4.7 Slowly Changing Dimension

DimGuest is implemented as a Slowly Changing Dimension Type 2. This means that when the country or customer type of a guest changes over time, the existing record is not overwritten. Instead, the current record is expired by setting the expiry_date to the date of the change and the is_current flag to 0, and a new record is inserted with the updated values, a new effective_date and the is_current flag set to 1. This approach preserves the full history of guest attribute changes, allowing accurate historical analysis in the data warehouse.

4.8 Assumptions

The following assumptions were made during the design of the dimensional model. The arrival date is constructed by combining the arrival_date_year, arrival_date_month and arrival_date_day_of_month columns from the source data during the ETL process. The total_stays measure is derived by adding stays_weekend_nights and stays_week_nights, and total_revenue is derived by multiplying adr by total_stays. Room hierarchy attributes such as room_category and room_class are not available in the original dataset and are manually assigned based on the room type code. Null values in the agent_id column indicate direct bookings made without a booking agent. All surrogate keys are generated automatically using the IDENTITY property in SQL Server.

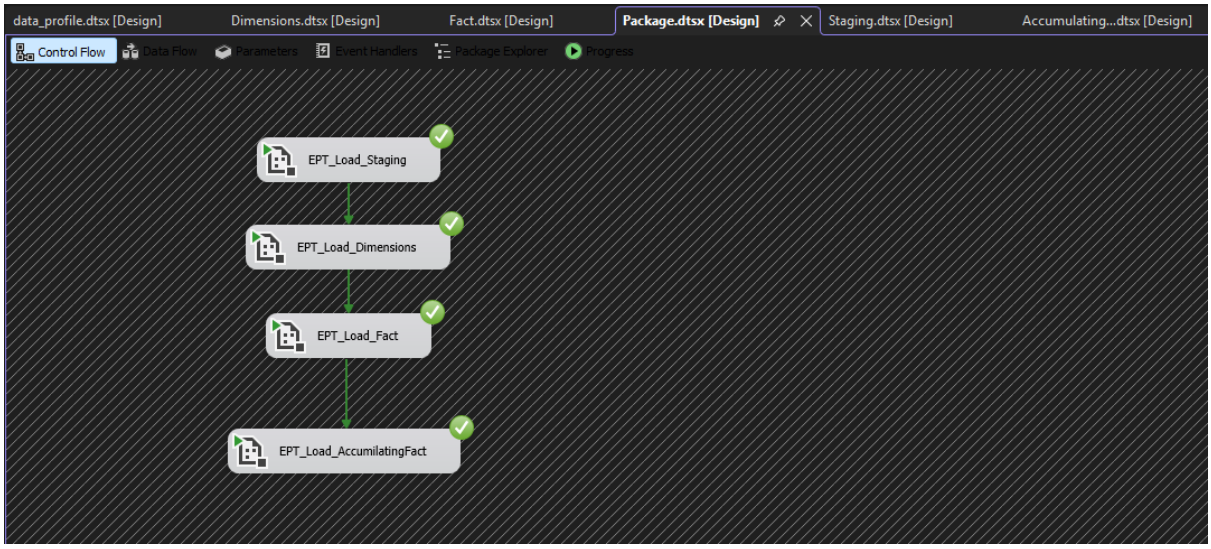
5. Task 5 - ETL Development

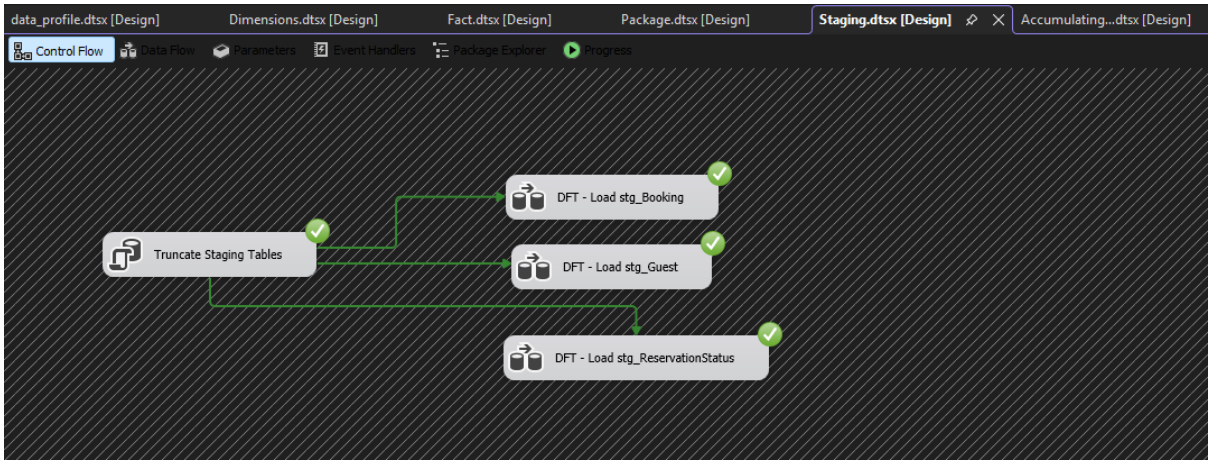
5.1 ETL Process Overview

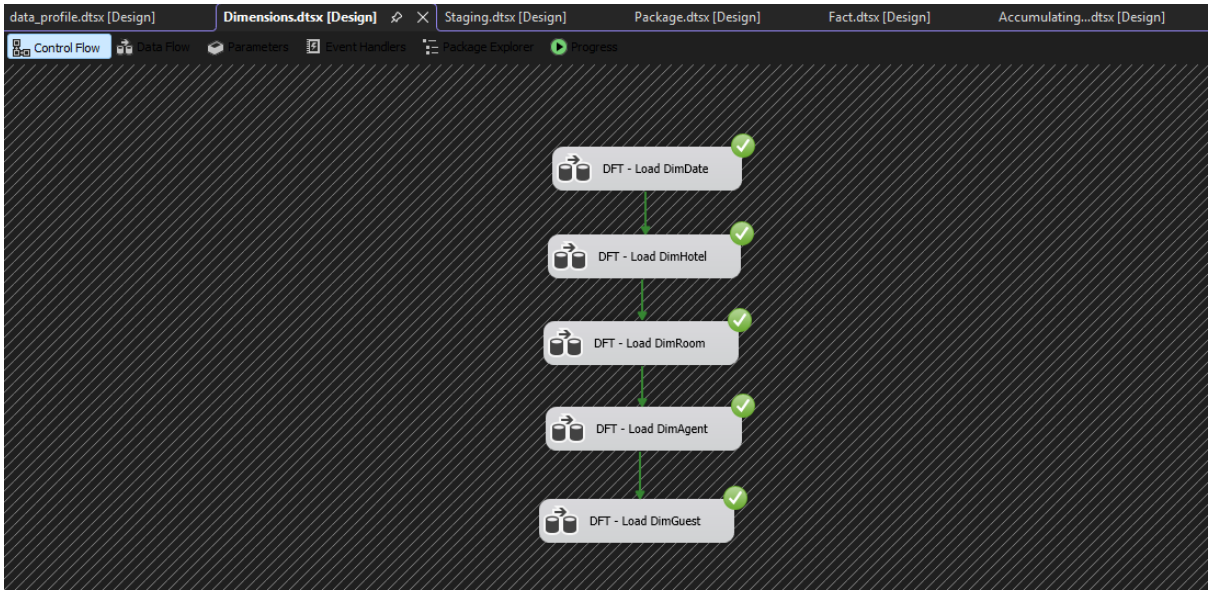
The ETL process is implemented using Microsoft SQL Server Integration Services (SSIS). The design follows a **layered approach** consisting of:

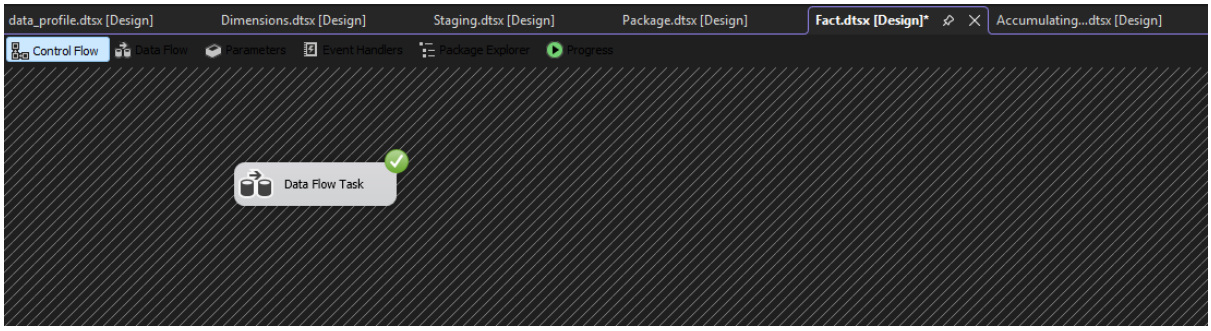
- Staging Layer
- Dimension Load Layer
- Fact Load Layer
- Master Control Package

A central SSIS package (Package.dtsx) is used to orchestrate all ETL workflows in a controlled sequence.


data_profile.dtsx [Design]
Dimensions.dtsx [Design]
Fact.dtsx [Design]
Package.dtsx [Design]
Staging.dtsx [Design]
Accumulating...dtsx [Design]
Control Flow
Data Flow
Parameters
Event Handlers
Package Explorer
Progress
EPT_Load_Staging
EPT_Load_Dimensions
EPT_Load_Fact
EPT_Load_AccumilatingFact


data_profile.dtsx [Design]
Dimensions.dtsx [Design]
Fact.dtsx [Design]
Package.dtsx [Design]
Staging.dtsx [Design]
Accumulating...dtsx [Design]
Control Flow
Data Flow
Parameters
Event Handlers
Package Explorer
Progress
Truncate Staging Tables
DFT - Load stg_Booking
DFT - Load stg_Guest
DFT - Load stg_ReservationStatus


data_profile.dtsx [Design]
Dimensions.dtsx [Design]
Staging.dtsx [Design]
Package.dtsx [Design]
Fact.dtsx [Design]
Accumulating...dtsx [Design]
Control Flow
Data Flow
Parameters
Event Handlers
Package Explorer
Progress
DFT - Load DimDate
DFT - Load DimHotel
DFT - Load DimRoom
DFT - Load DimAgent
DFT - Load DimGuest



5.2 Data Profiling Task

Data Profiling Task in SSIS is used to analyze the quality, structure, and consistency of source data before ETL processing.

Implementation:

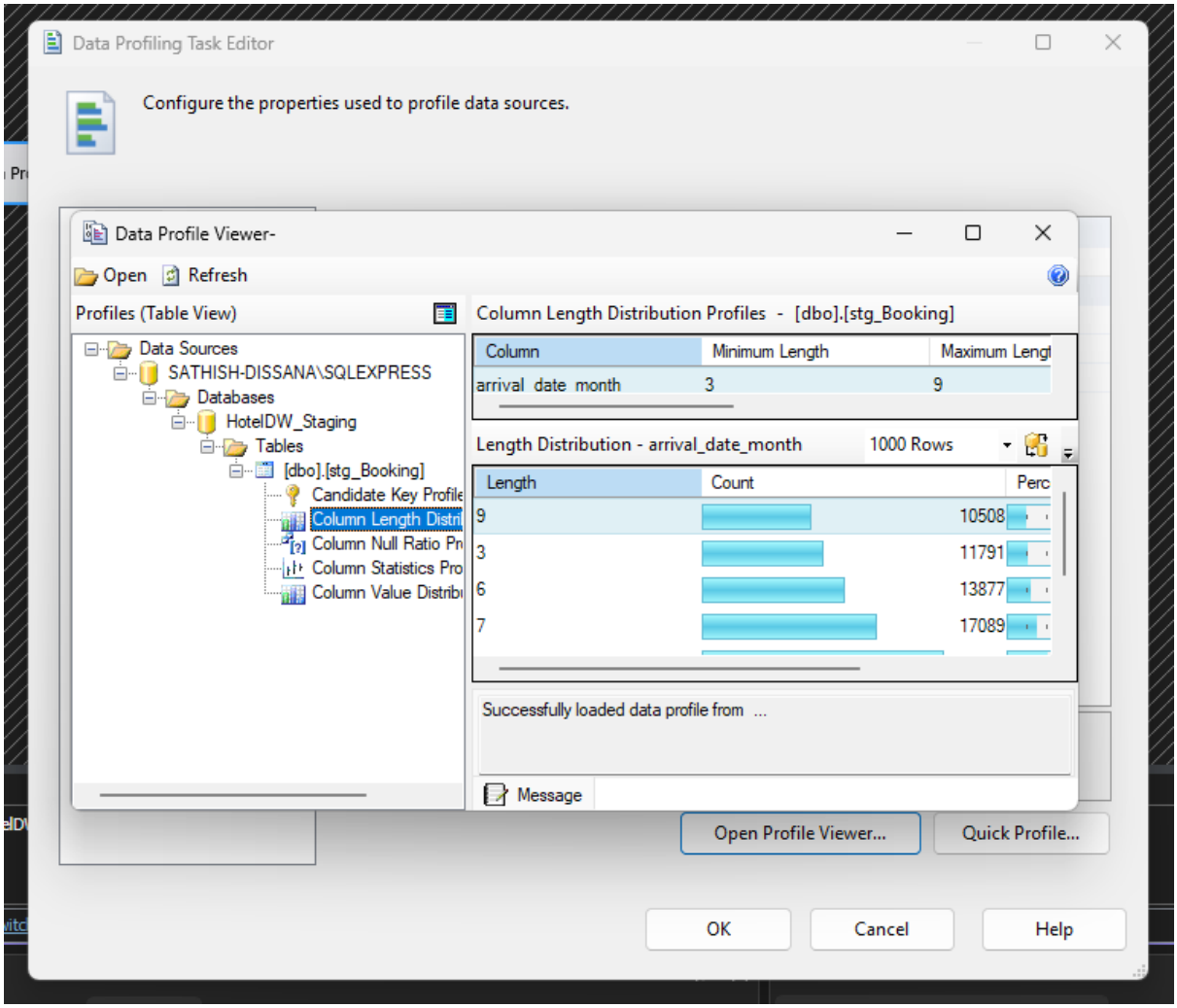
A Data Profiling Task was executed on the source data to identify:

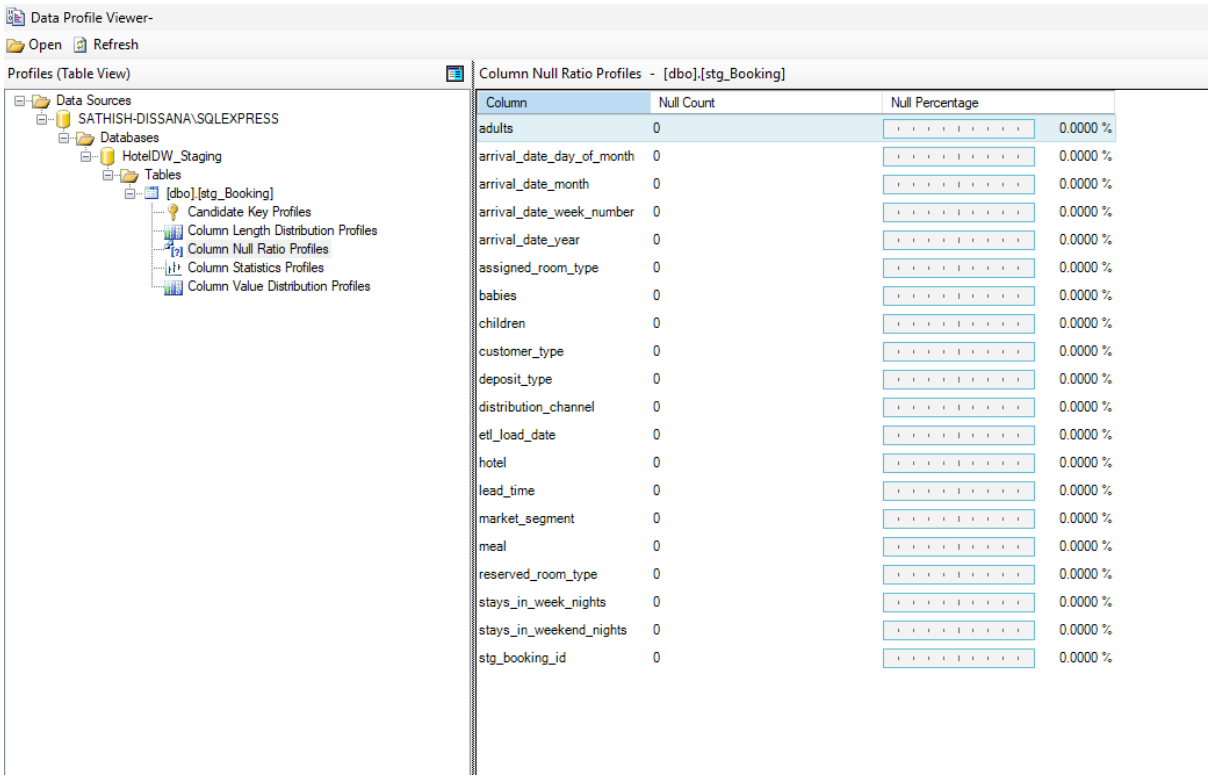
- Null value distribution
- Data type consistency
- Value frequency distribution
- Uniqueness of key columns
- Data anomalies and inconsistencies

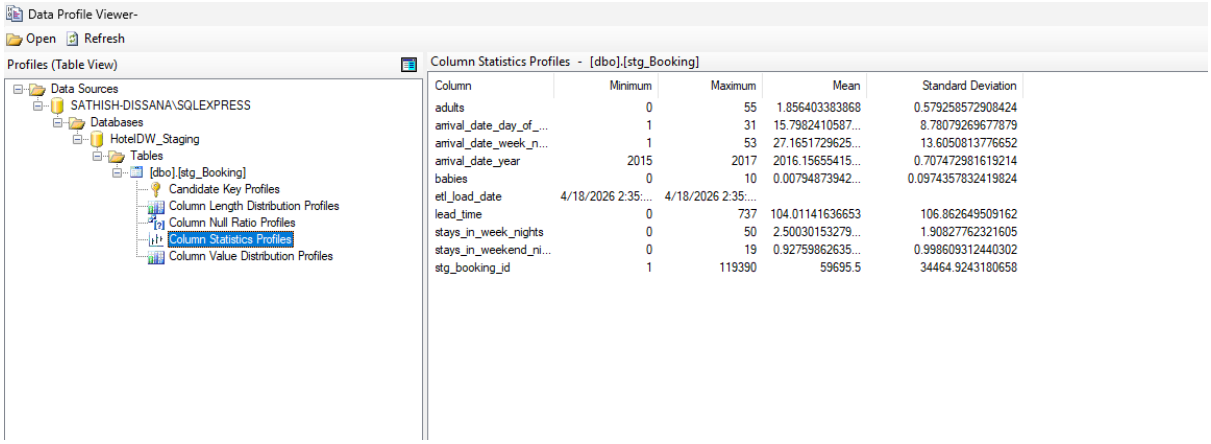
Outcome:

The profiling results confirmed that:

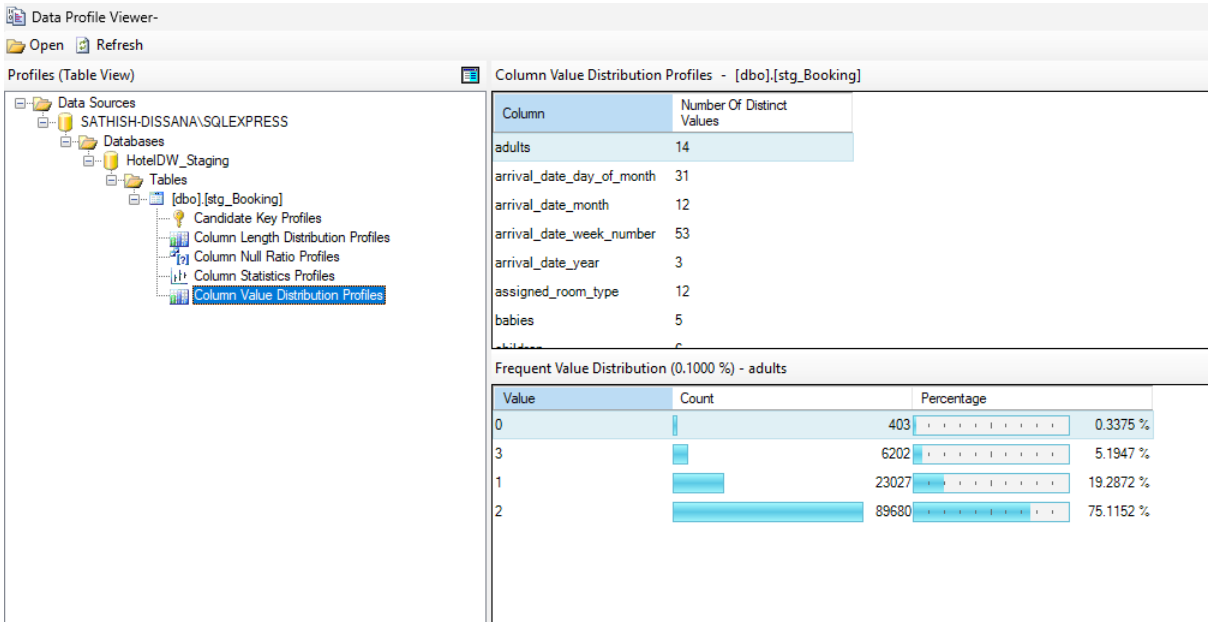
- Booking and guest data are mostly complete
- No major structural inconsistencies were found
- Month values required standardization (string format handling)
- Suitable for ETL processing after staging load


Data Profiling Task Editor
Configure the properties used to profile data sources.
Data Profile Viewer-
Open
Refresh
Profiles (Table View)
Data Sources
SATHISH-DISSANA\SQLEXPRESS
Databases
HotelDW_Staging
Tables
[dbo].[stg_Booking]
Candidate Key Profile
Column Length Distri
Column Null Ratio Pr
Column Statistics Pro
Column Value Distribu
Column Length Distribution Profiles - [dbo].[stg_Booking]
Column
Minimum Length
Maximum Lengt
arrival_date_month
3
9
Length Distribution - arrival_date_month
1000 Rows
Length
Count
Perc
9
10508
3
11791
6
13877
7
17089
Successfully loaded data profile from ...
Message
Open Profile Viewer...
Quick Profile...
OK
Cancel
Help


Data Profile Viewer-
Open
Refresh
Profiles (Table View)
Data Sources
SATHISH-DISSANA\SQLEXPRESS
Databases
HotelDW_Staging
Tables
[dbo].[stg_Booking]
Candidate Key Profiles
Column Length Distribution Profiles
Column Null Ratio Profiles
Column Statistics Profiles
Column Value Distribution Profiles
Column Null Ratio Profiles - [dbo].[stg_Booking]
Column
Null Count
Null Percentage
adults 0 0.0000 %
arrival_date_day_of_month 0 0.0000 %
arrival_date_month 0 0.0000 %
arrival_date_week_number 0 0.0000 %
arrival_date_year 0 0.0000 %
assigned_room_type 0 0.0000 %
babies 0 0.0000 %
children 0 0.0000 %
customer_type 0 0.0000 %
deposit_type 0 0.0000 %
distribution_channel 0 0.0000 %
etl_load_date 0 0.0000 %
hotel 0 0.0000 %
lead_time 0 0.0000 %
market_segment 0 0.0000 %
meal 0 0.0000 %
reserved_room_type 0 0.0000 %
stays_in_week_nights 0 0.0000 %
stays_in_weekend_nights 0 0.0000 %
stg_booking_id 0 0.0000 %



Column	Minimum	Maximum	Mean	Standard Deviation
adults	0	55	1.856403383868	0.579258572908424
arrival_date_day_of_...	1	31	15.7982410587...	8.78079269677879
arrival_date_week_n...	1	53	27.1651729625...	13.6050813776652
arrival_date_year	2015	2017	2016.15655415...	0.707472981619214
babies	0	10	0.00794873942...	0.0974357832419824
etl_load_date	4/18/2026 2:35:...	4/18/2026 2:35:...		
lead_time	0	737	104.01141636653	106.862649509162
stays_in_week_nights	0	50	2.50030153279...	1.90827762321605
stays_in_weekend_ni...	0	19	0.92759862635...	0.998609312440302
stg_booking_id	1	119390	59695.5	34464.9243180658



Column	Number Of Distinct Values
adults	14
arrival_date_day_of_month	31
arrival_date_month	12
arrival_date_week_number	53
arrival_date_year	3
assigned_room_type	12
babies	5

Frequent Value Distribution (0.1000 %) - adults

Value	Count	Percentage
0	403	0.3375 %
3	6202	5.1947 %
1	23027	19.2872 %
2	89680	75.1152 %

5.3 Staging Layer (staging.dtsx)

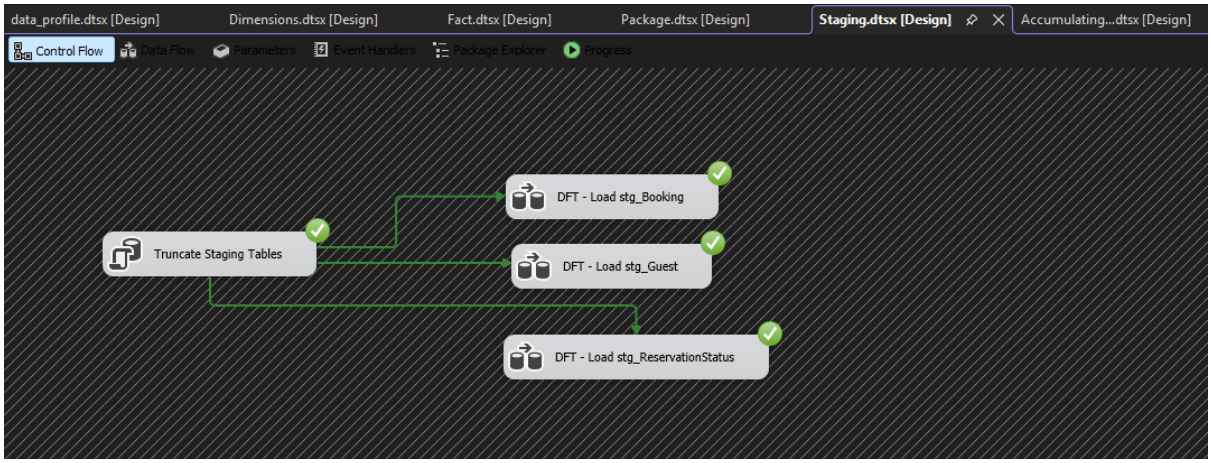
To extract raw data from multiple sources and load into staging tables without transformation.

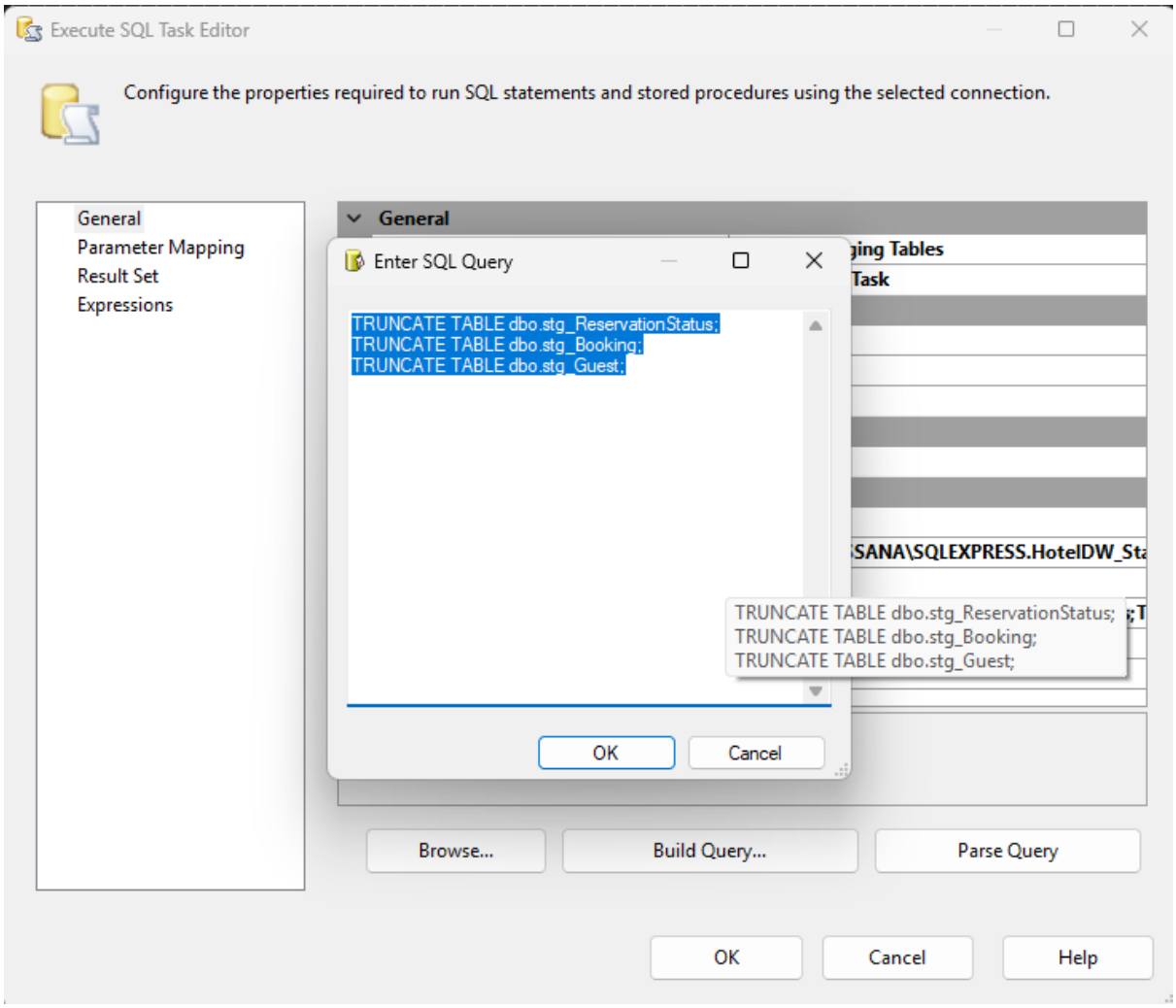
Process Steps:

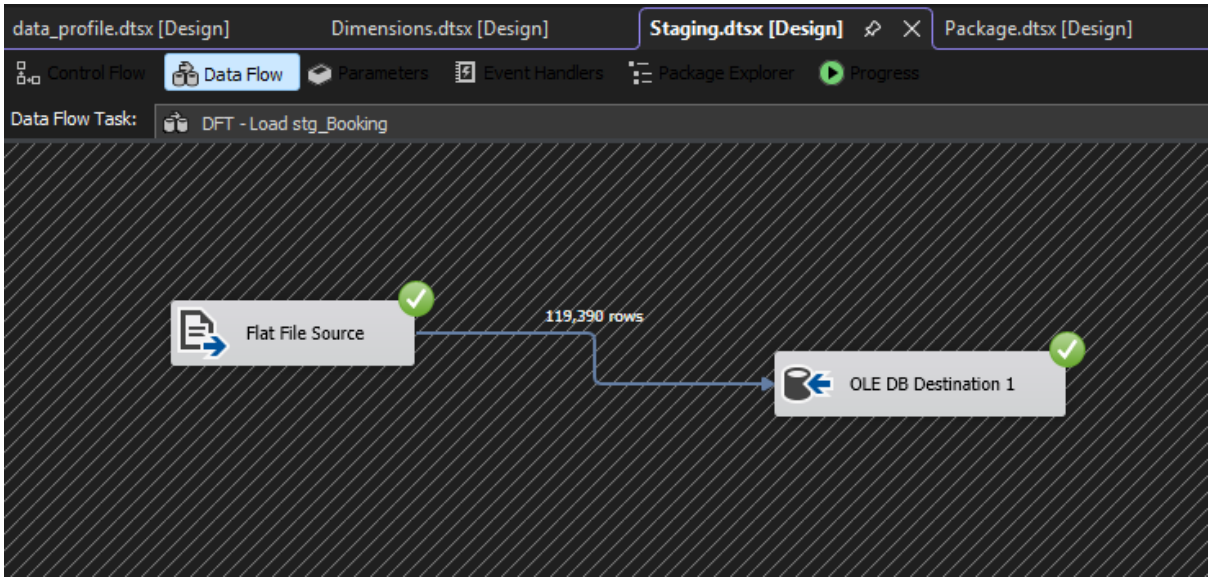
1. Truncate staging tables using Execute SQL Task
2. Load data into staging tables using Data Flow Tasks

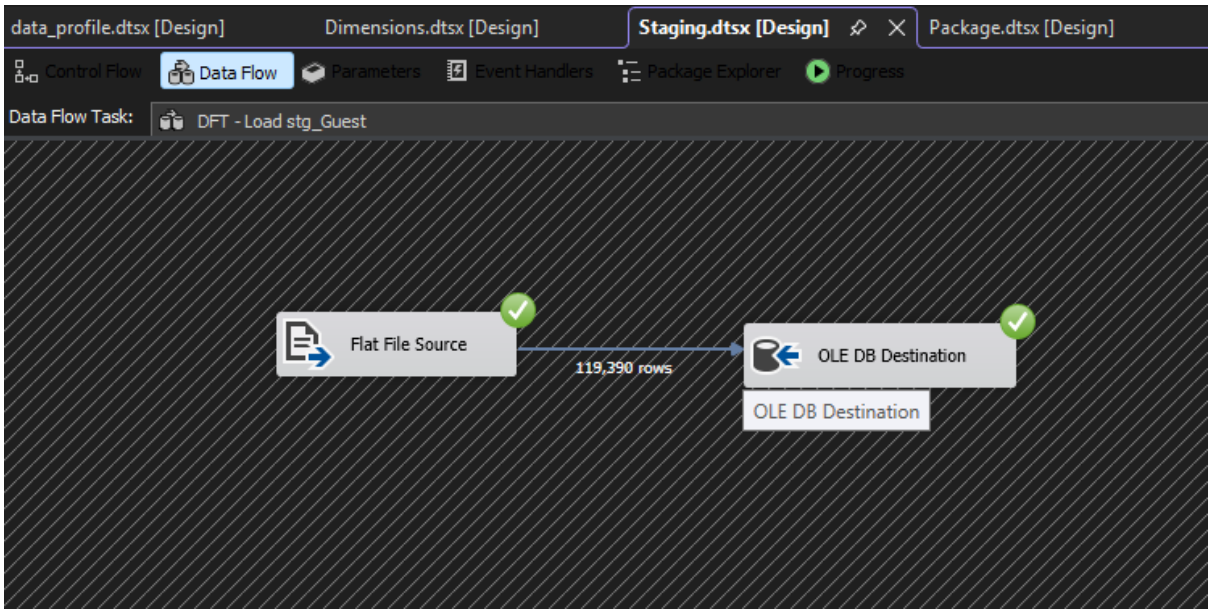
Data Flow:

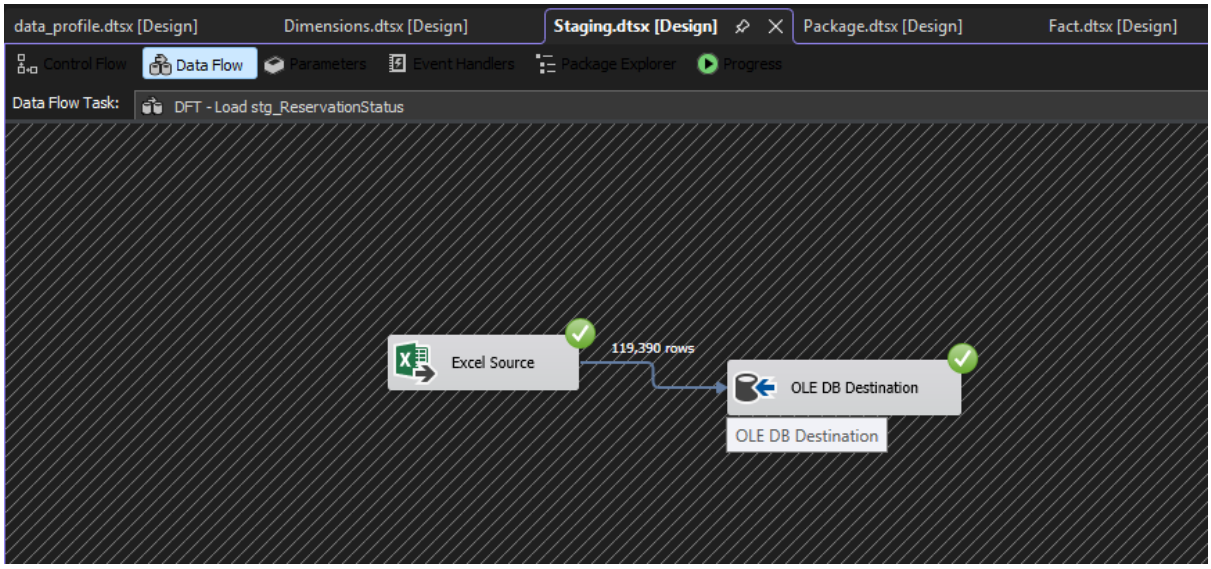
- Source → Flat File Source / Excel Source
- Destination → stg_Booking, stg_Guest, stg_Reservation_status


data_profile.dtsx [Design]
Dimensions.dtsx [Design]
Fact.dtsx [Design]
Package.dtsx [Design]
Staging.dtsx [Design]
Accumulating...dtsx [Design]
Control Flow
Data Flow
Parameters
Event Handlers
Package Explorer
Progress
Truncate Staging Tables
DFT - Load stg_Booking
DFT - Load stg_Guest
DFT - Load stg_ReservationStatus

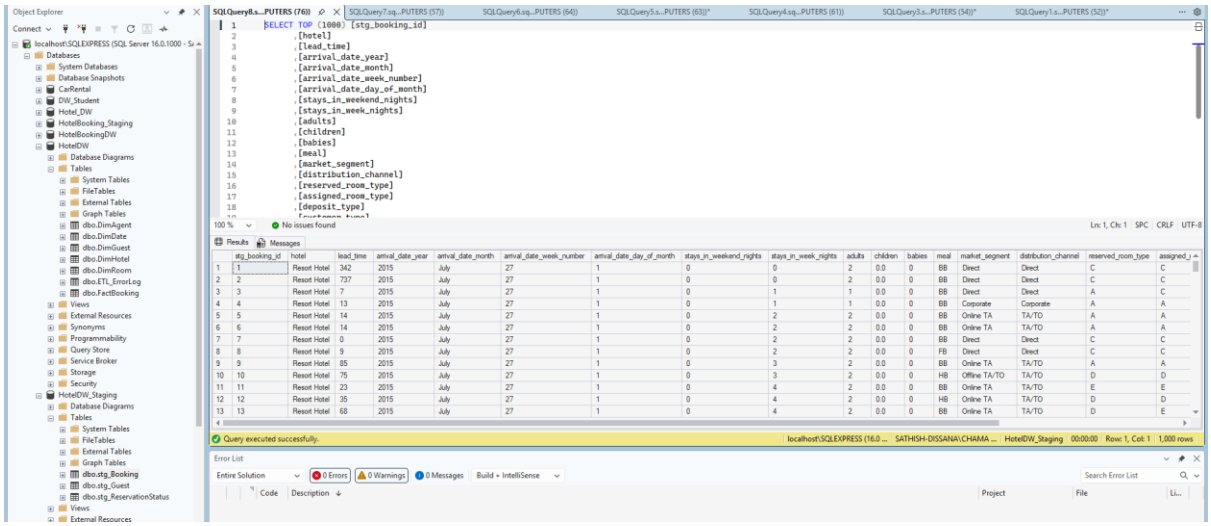

Execute SQL Task Editor
Configure the properties required to run SQL statements and stored procedures using the selected connection.
General
Parameter Mapping
Result Set
Expressions
General
Enter SQL Query
TRUNCATE TABLE dbo.stg_ReservationStatus;
TRUNCATE TABLE dbo.stg_Booking;
TRUNCATE TABLE dbo.stg_Guest;
OK
Cancel
TRUNCATE TABLE dbo.stg_ReservationStatus;
TRUNCATE TABLE dbo.stg_Booking;
TRUNCATE TABLE dbo.stg_Guest;
Browse...
Build Query...
Parse Query
OK
Cancel
Help

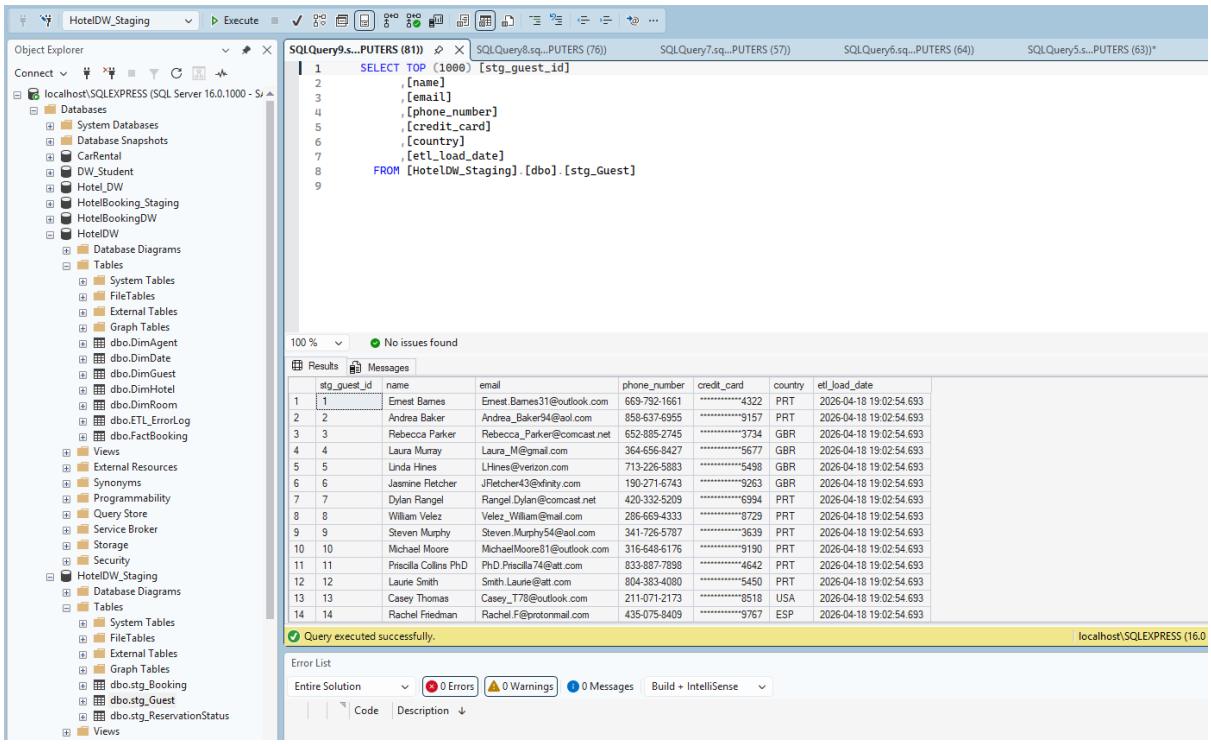

data_profile.dtsx [Design]
Dimensions.dtsx [Design]
Staging.dtsx [Design]
Package.dtsx [Design]
Control Flow
Data Flow
Parameters
Event Handlers
Package Explorer
Progress
Data Flow Task:
DFT - Load stg_Booking
Flat File Source
119,390 rows
OLE DB Destination 1

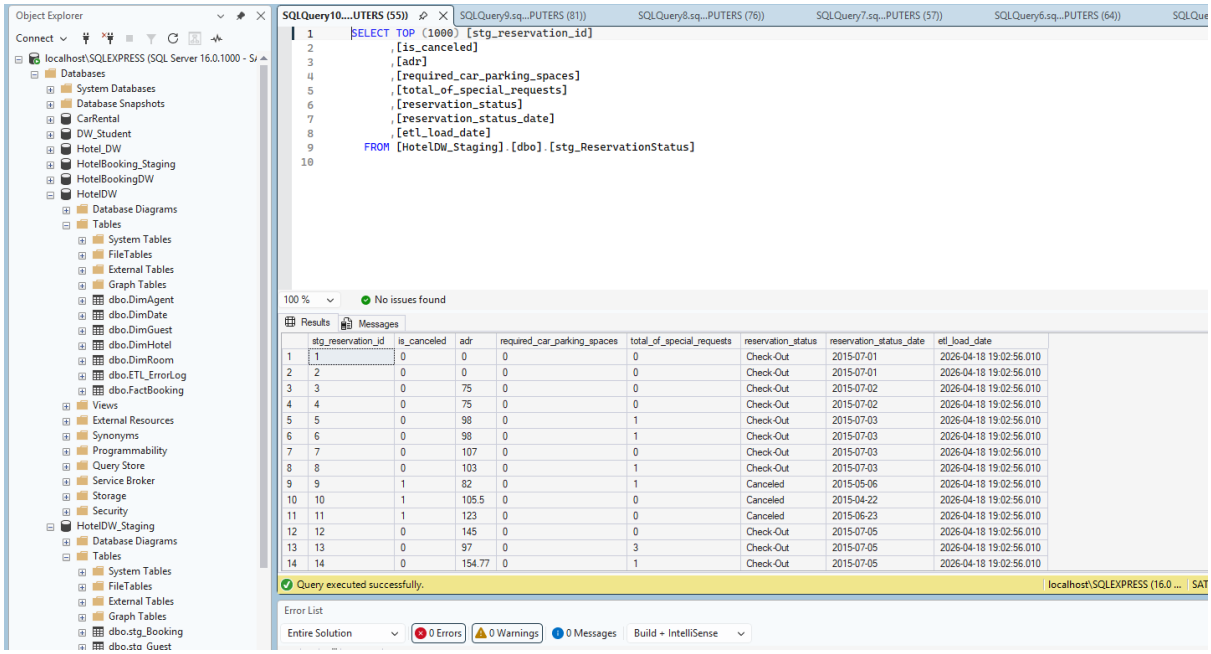

data_profile.dtsx [Design]
Dimensions.dtsx [Design]
Staging.dtsx [Design]
Package.dtsx [Design]
Control Flow
Data Flow
Parameters
Event Handlers
Package Explorer
Progress
Data Flow Task:
DFT - Load stg_Guest
Flat File Source
119,390 rows
OLE DB Destination
OLE DB Destination


data_profile.dtsx [Design]
Dimensions.dtsx [Design]
Staging.dtsx [Design]
Package.dtsx [Design]
Fact.dtsx [Design]
Control Flow
Data Flow
Parameters
Event Handlers
Package Explorer
Progress
Data Flow Task:
DFT - Load stg_ReservationStatus
Excel Source
119,390 rows
OLE DB Destination
OLE DB Destination

5.4 Validation(Staging tables loaded to SSMS):







5.4 Dimension Load Layer (Dimensions.dtsx)

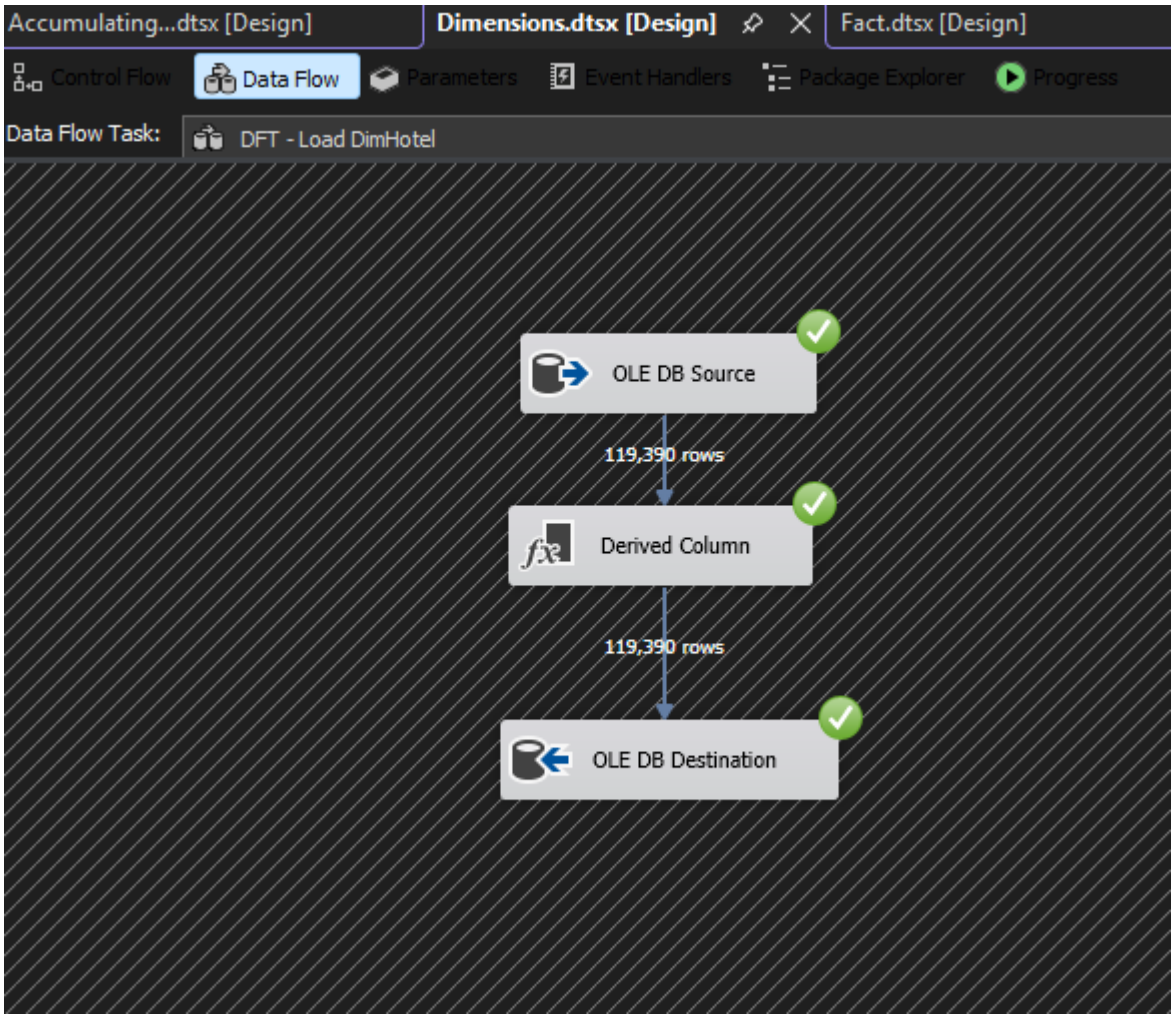
Purpose:

To load dimension tables required for the data warehouse from staging data

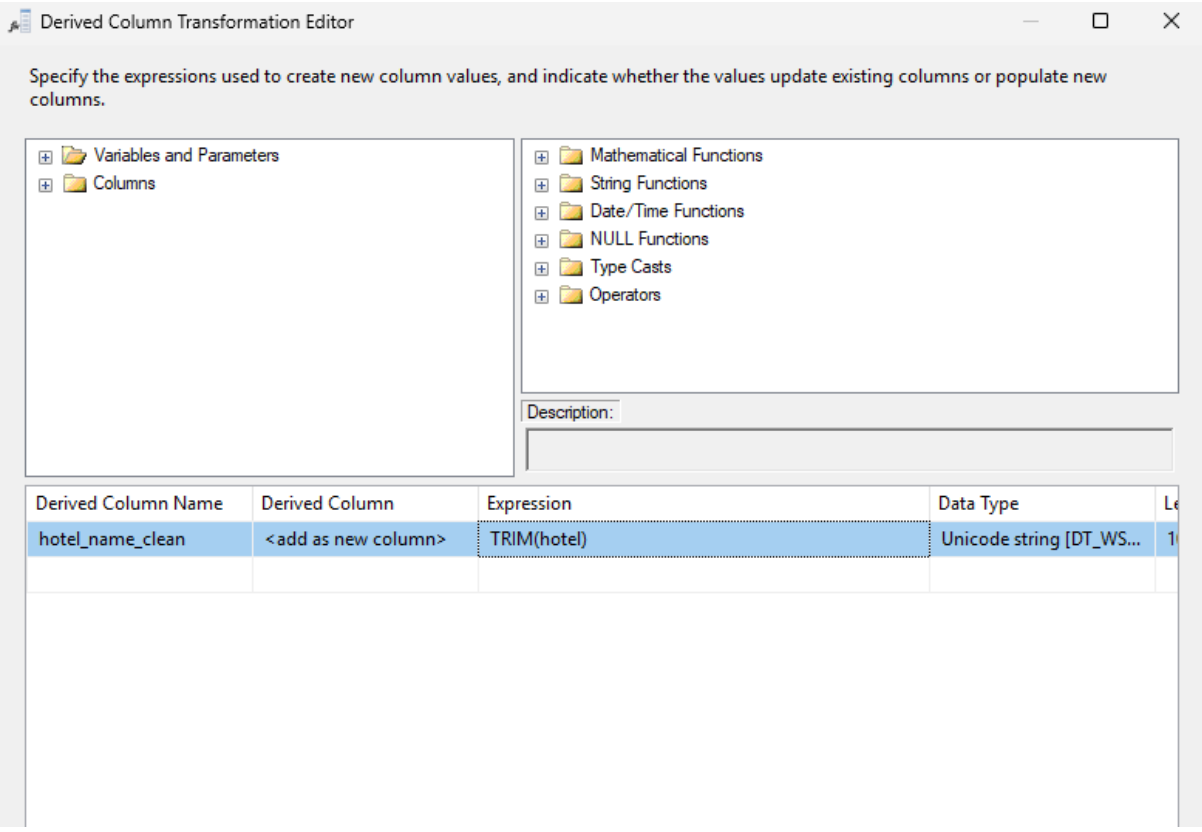
Execution Order:

1. DimHotel
2. DimRoom
3. DimAgent
4. DimGuest (SCD Type 2)
5. DimDate

5.4.1 DimHotel

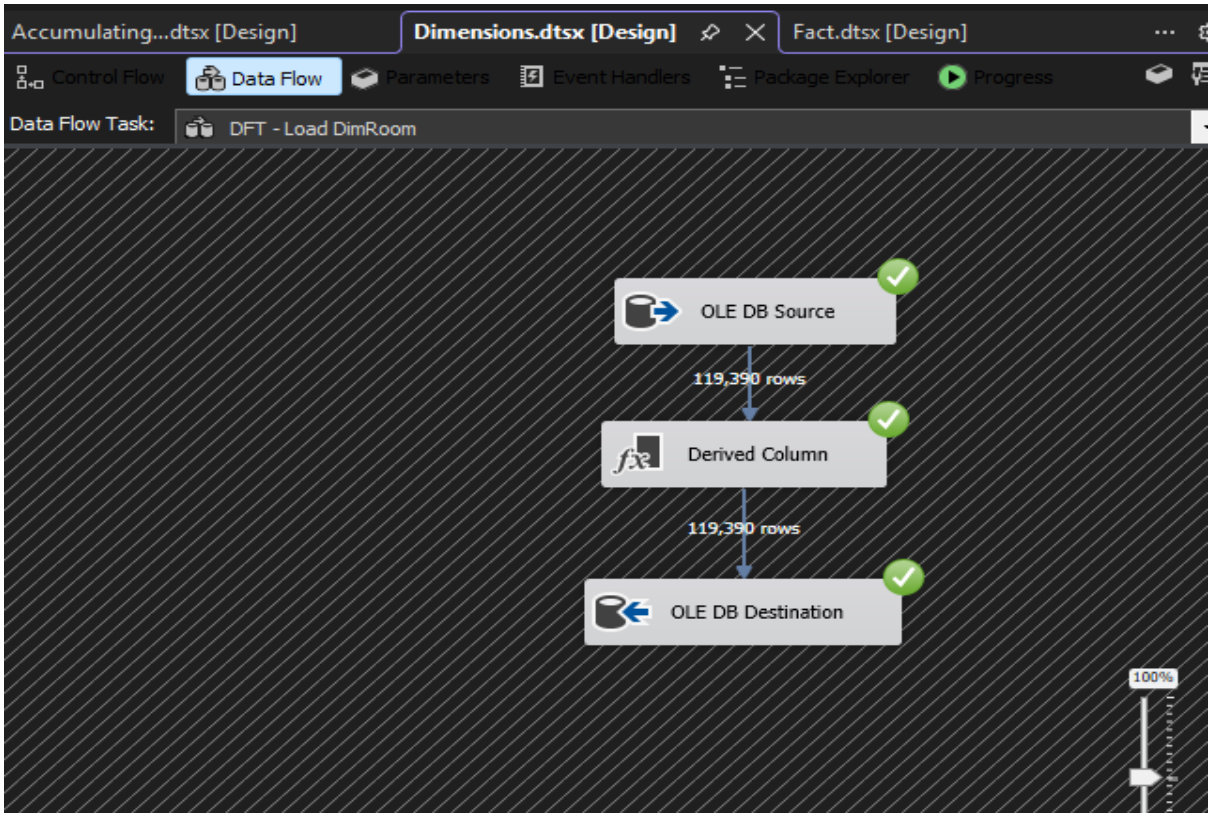


A Derived Column transformation was applied to clean hotel attribute values using the TRIM function to remove leading and trailing spaces, ensuring data consistency.

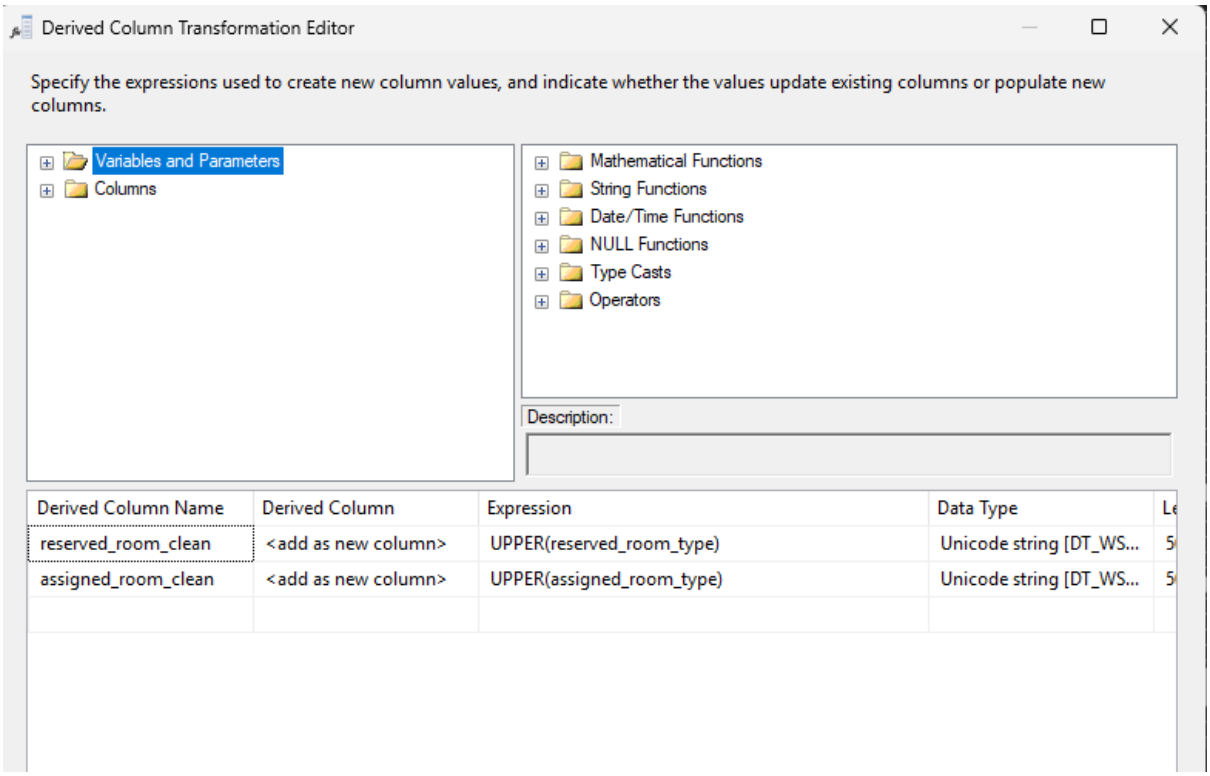


5.4.2 DimRoom

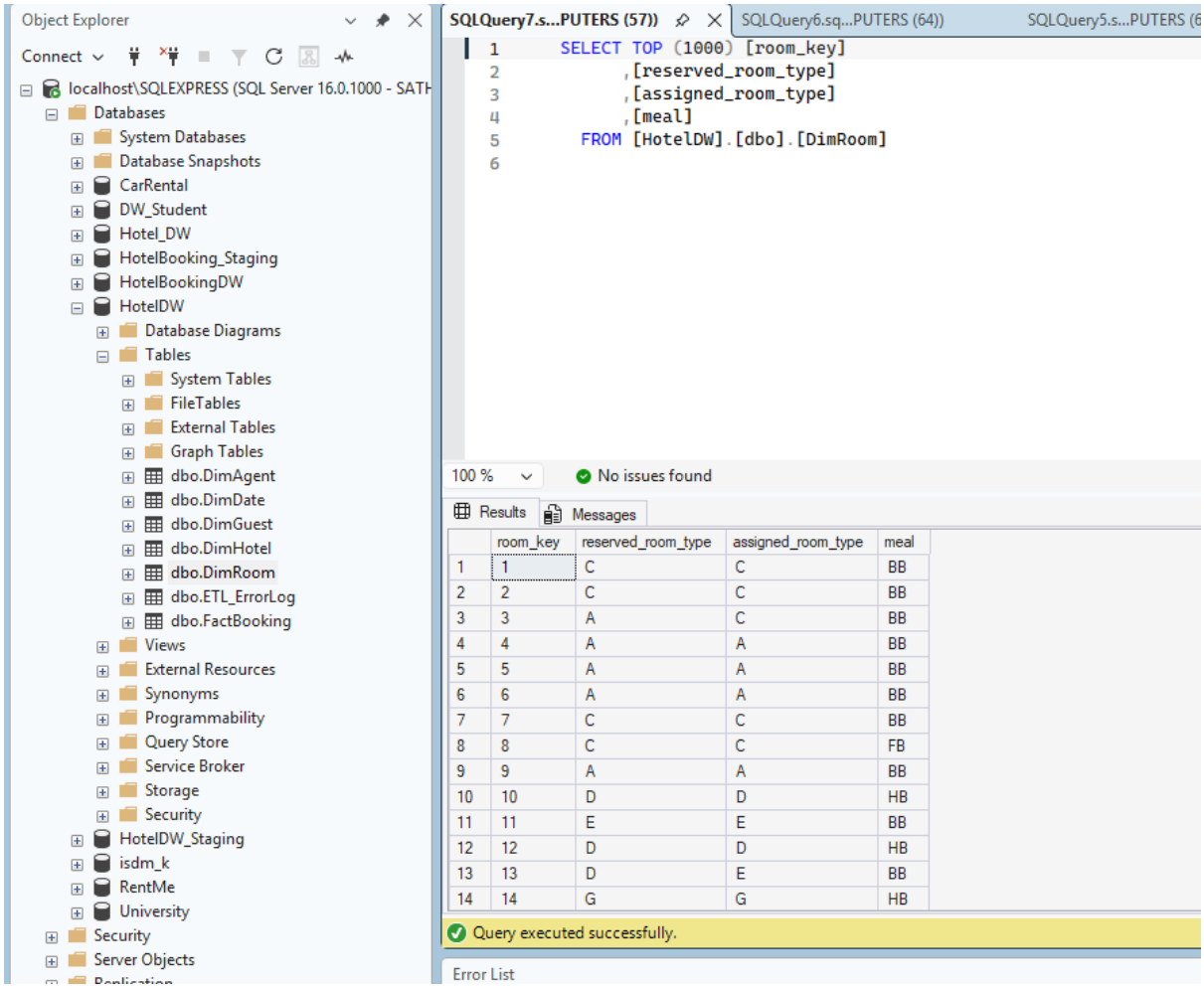
Room-related attributes are extracted from staging booking data and loaded into the dimension table.



A Derived Column transformation was used to standardize room type attributes by converting text values to uppercase for consistency in analysis.

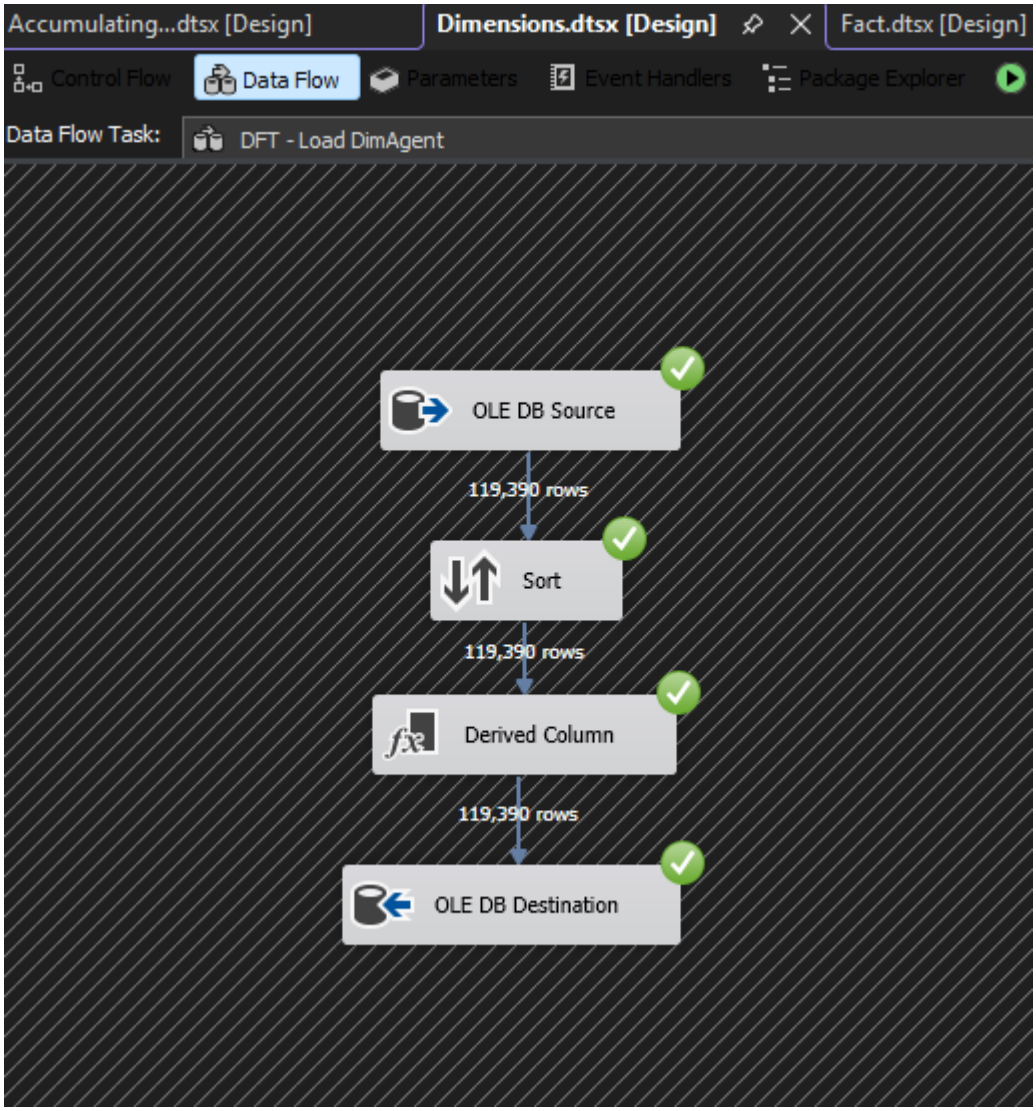


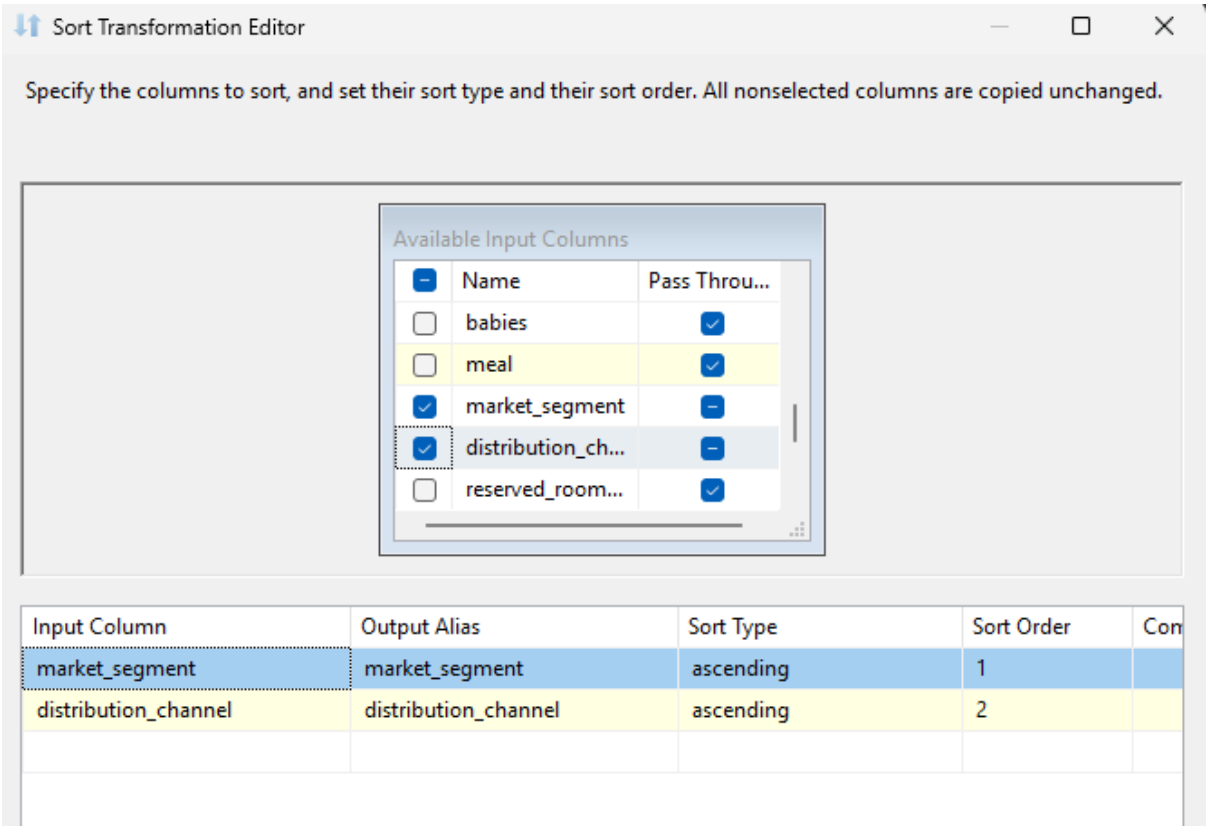
Validation (Data Added to SSMS) :

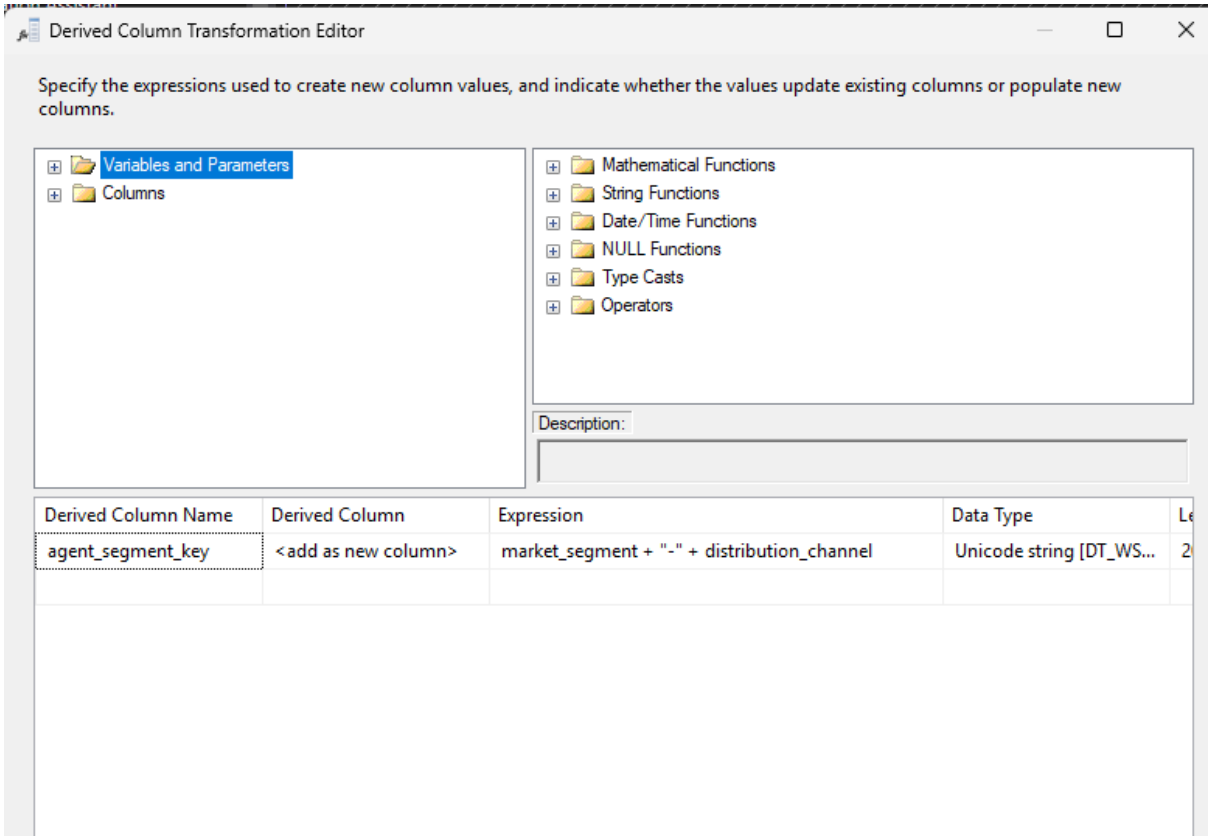


5.4.3 DimAgent

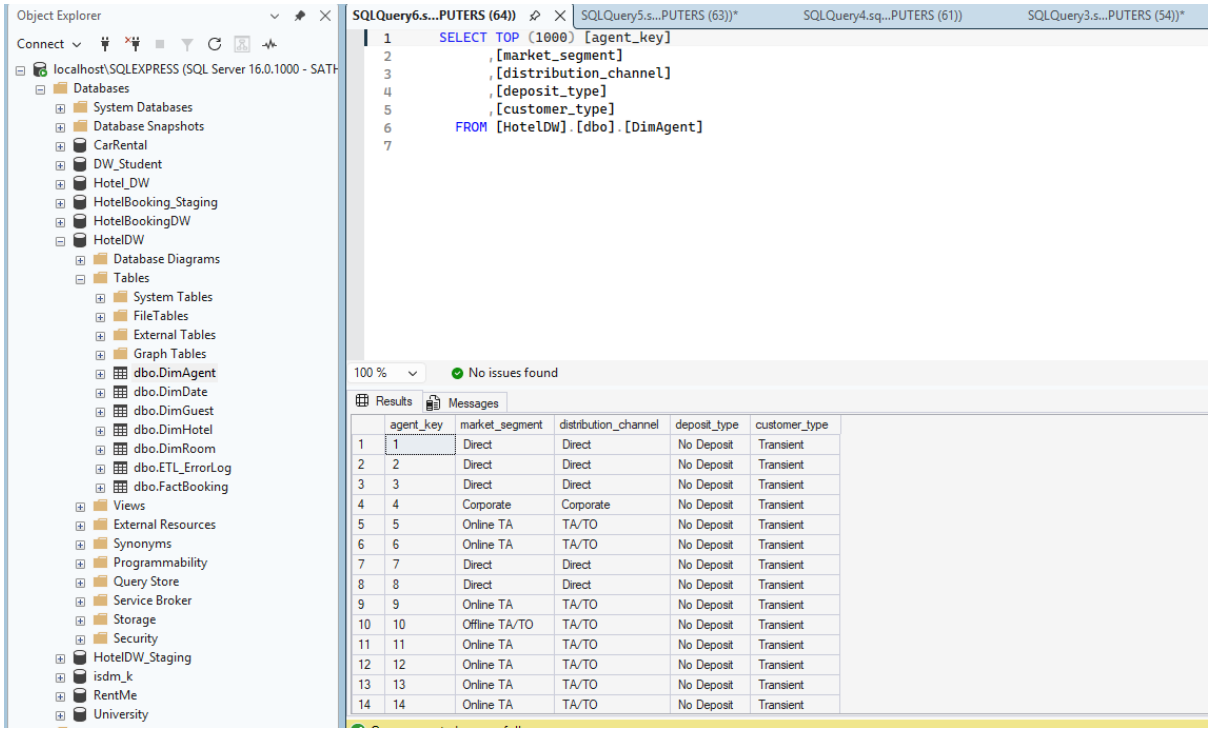
A Sort transformation was applied to organize records by market segment and distribution channel. Additionally, a Derived Column transformation was used to generate a composite attribute combining both fields for analytical grouping.


Accumulating...dtsx [Design]
Dimensions.dtsx [Design]
Fact.dtsx [Design]
Control Flow
Data Flow
Parameters
Event Handlers
Package Explorer
Data Flow Task:
DFT - Load DimAgent
OLE DB Source
119,390 rows
Sort
119,390 rows
Derived Column
119,390 rows
OLE DB Destination





Validation(Data Added to SSMS):

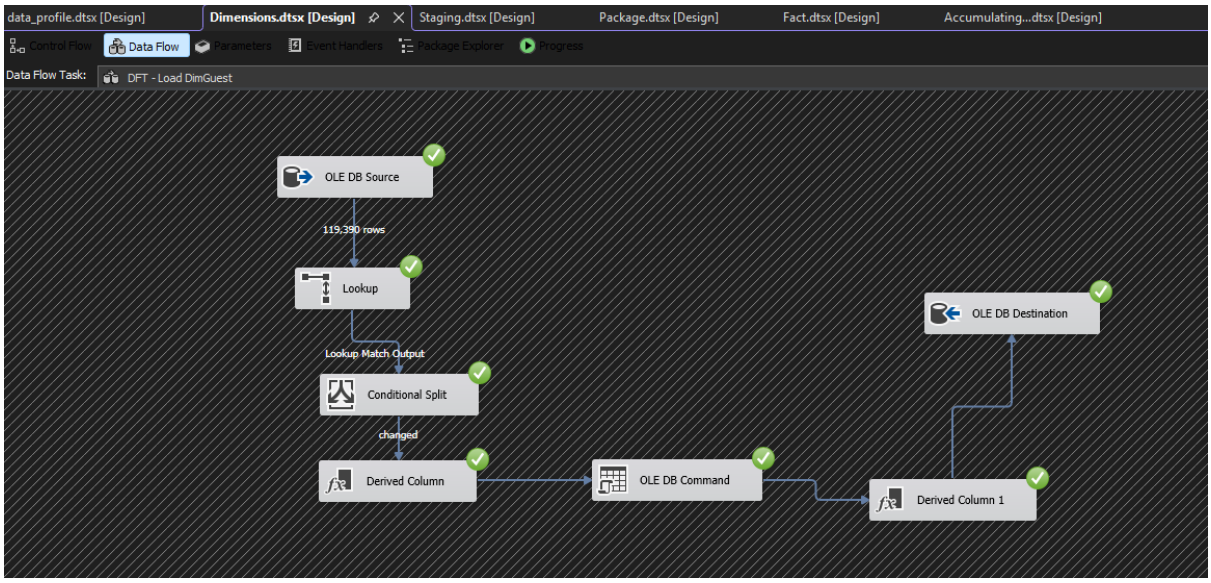


5.4.4 DimGuest (SCD Type 2 Implementation)

Purpose:

The DimGuest table is implemented using Slowly Changing Dimension Type 2 (SCD Type 2) to preserve historical changes of guest-related attributes such as name, email, phone number, credit card, and country.

This ensures that any changes in guest information are tracked over time without losing previous records.



1. Data Extraction (OLE DB Source)

Data is extracted from the staging table stg_Guest.

2. Lookup Transformation

A Lookup transformation is used to check whether the guest already exists in the DimGuest table.

Mapped Columns:

- guest_key
- name
- email
- phone_number
- credit_card
- country

This step identifies existing records and enables comparison between source and warehouse data.

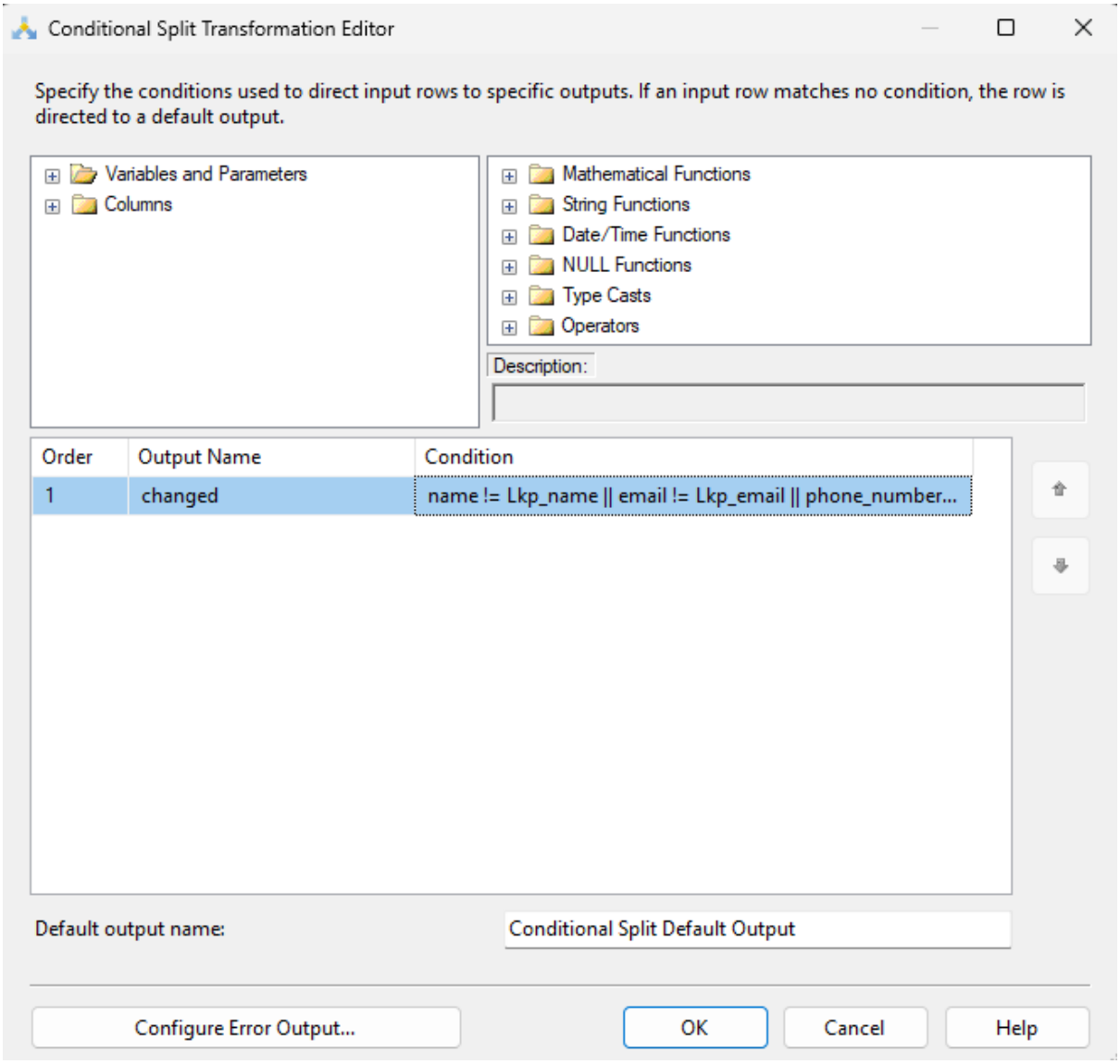
3. Conditional Split (Change Detection)

The Conditional Split transformation is used to detect changes in guest attributes.

Change Condition:

Records are considered changed if any of the following attributes differ:

- name
- email
- phone_number
- credit_card
- country



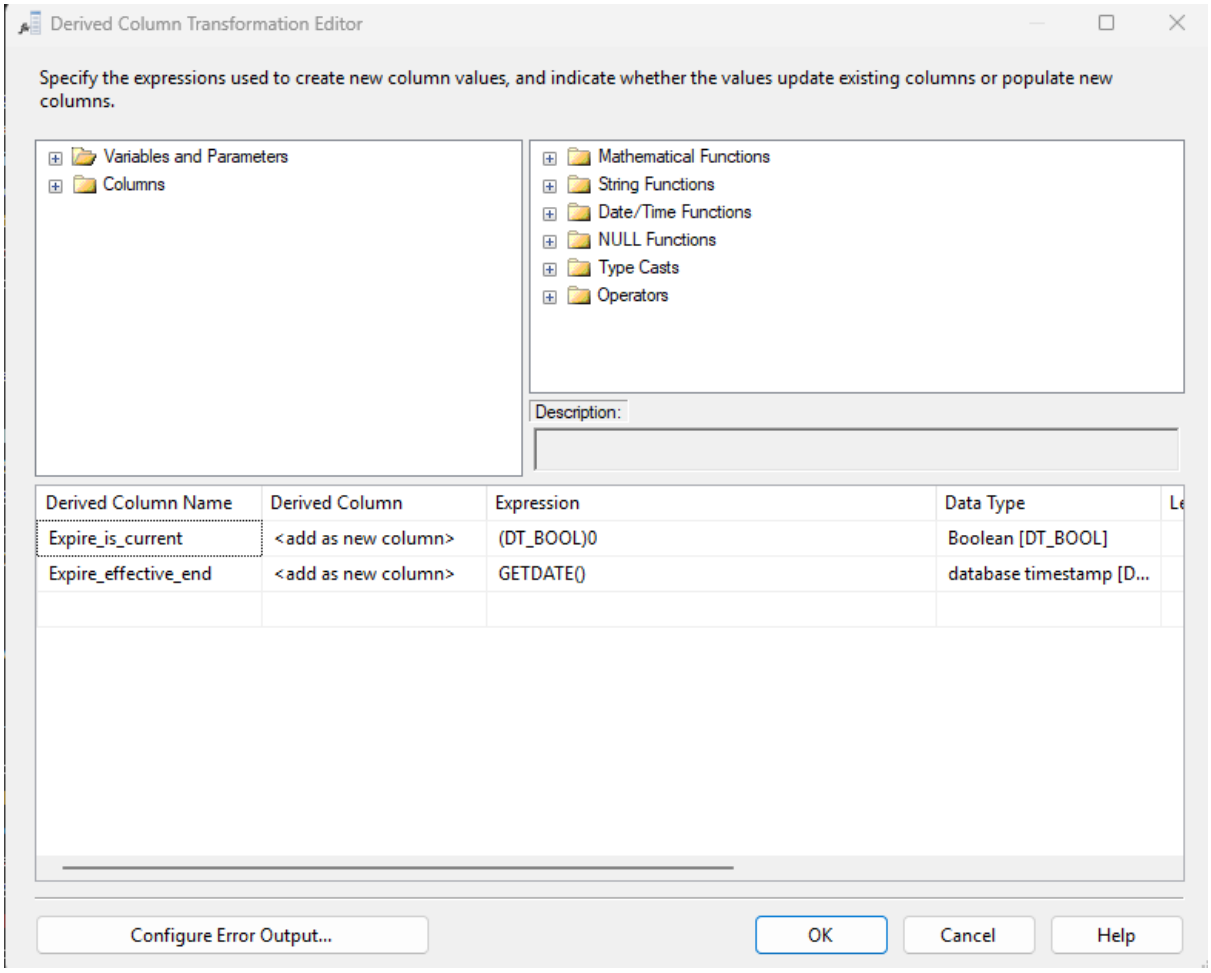
4.Expire Old Record (Derived Column + OLE DB Command):

For changed records, the existing record is marked as expired.

Derived Column Transformations:

- **Expire_is_current** = 0
- **Expire_effective_end** = GETDATE()

This step updates the existing record to mark it as inactive.



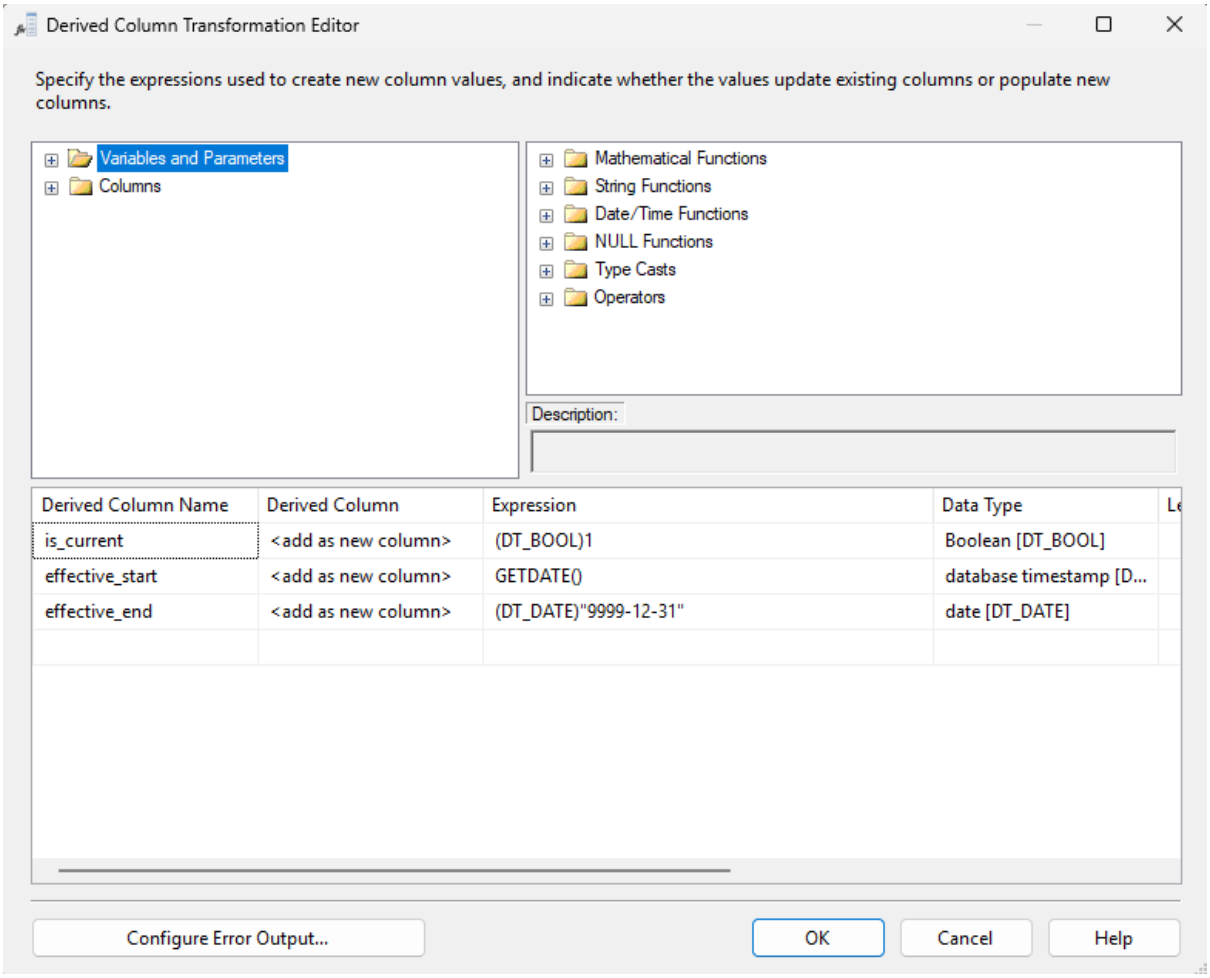
5. Insert New Version of Record

After expiring the old record, a new version of the guest record is created.

Derived Column Transformations:

- is_current = 1
- effective_start = GETDATE()
- effective_end = '9999-12-31'

This ensures that the latest record remains active while preserving historical data.



OLE DB Destination :

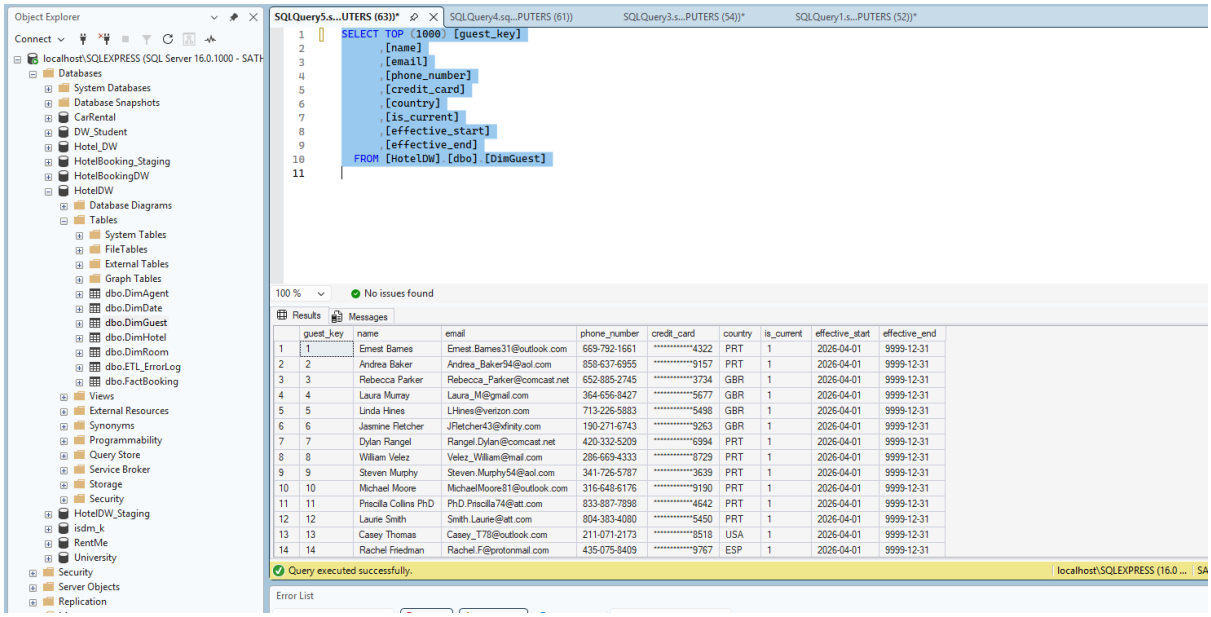
The final transformed data is inserted into the DimGuest table in the data warehouse.

Summary of SCD Type 2 Logic :

The implementation ensures:

- Historical tracking of guest changes
- Preservation of old and new records
- Accurate time-based analysis
- Proper use of SSIS transformations (Lookup, Conditional Split, Derived Column, OLE DB Command)
-

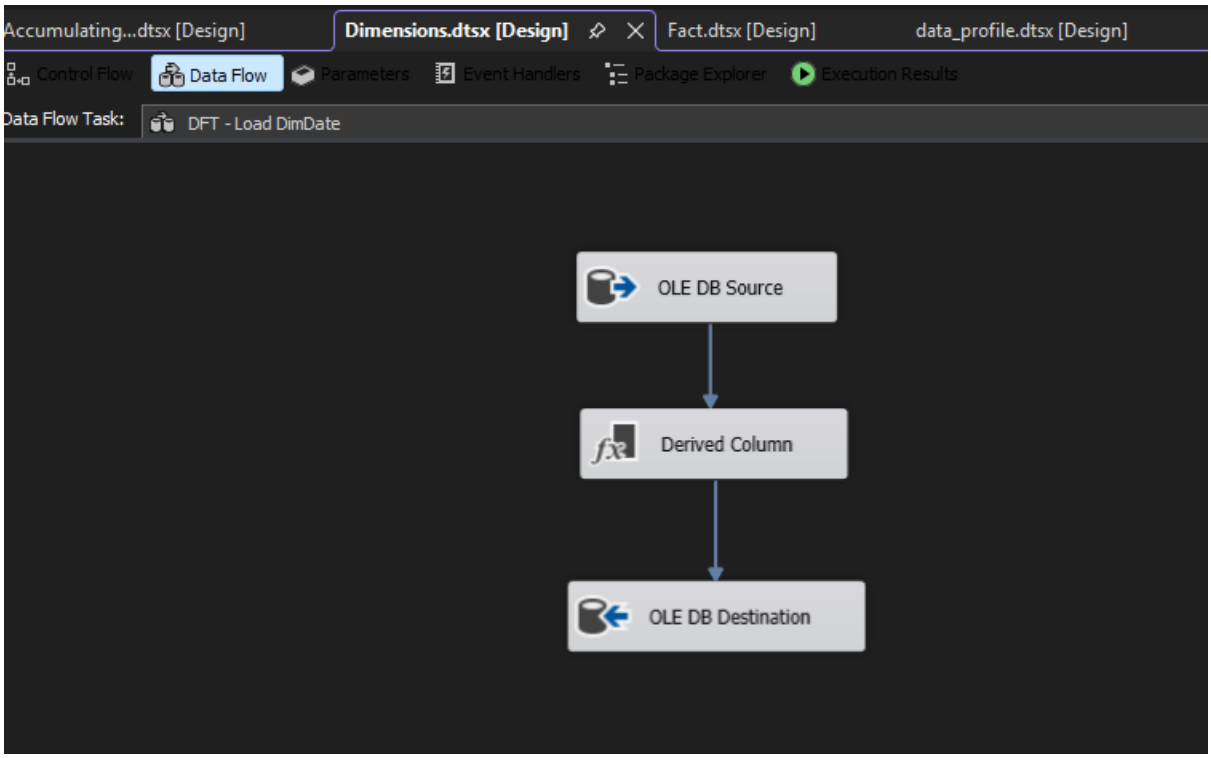
Validation(Data Loaded to SSMS):



5.4.5 DimDate

Purpose:

The primary purpose of the DimDate table is to provide a structured date hierarchy that allows efficient analysis of booking trends over time. It supports reporting requirements such as monthly booking patterns, seasonal trends, and yearly comparisons.



1. OLE DB Source

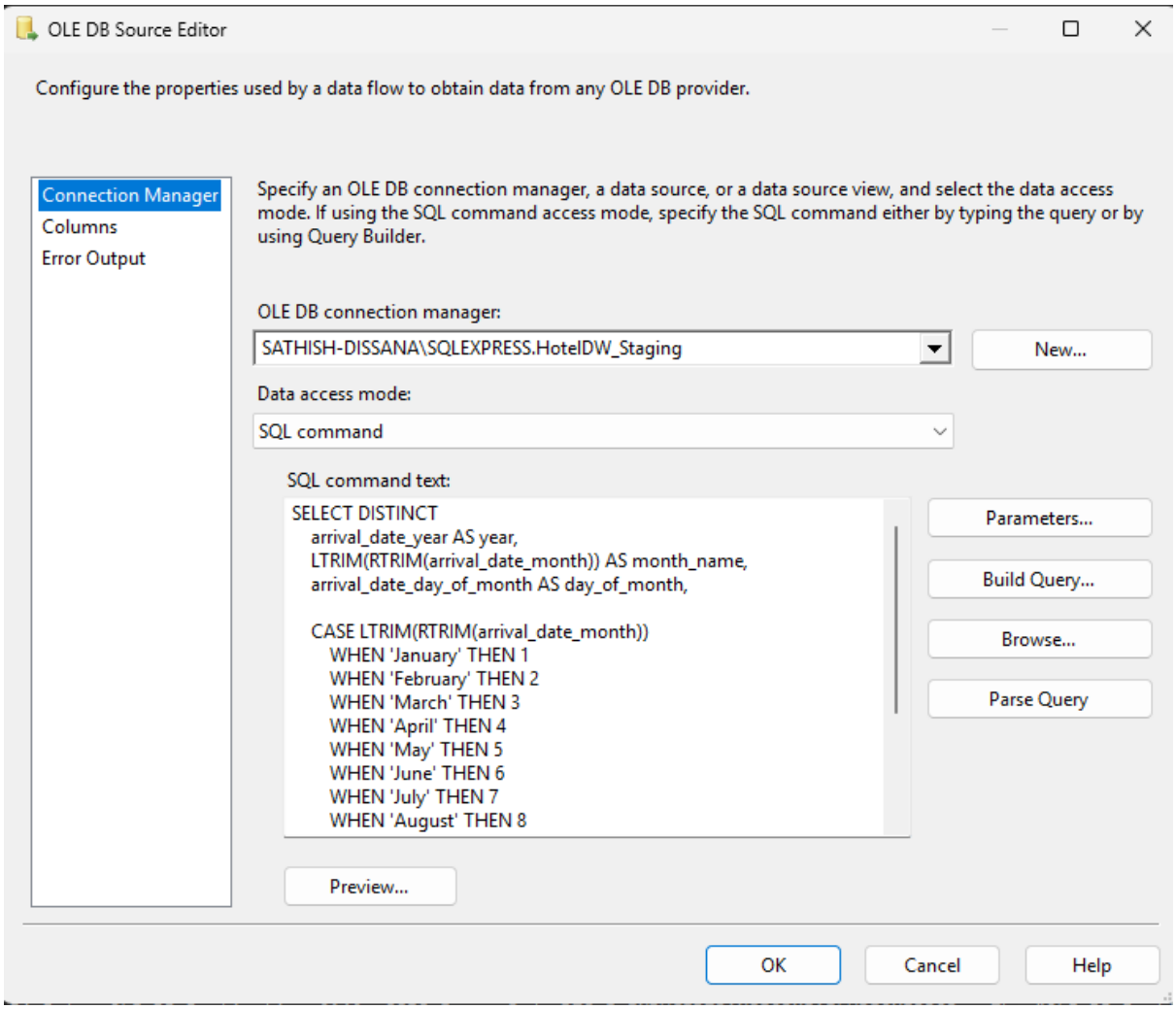
Data is extracted from the staging table (stg_Booking). Duplicate records are removed using the DISTINCT keyword to ensure uniqueness.

2. Data Standardization (SQL Transformation)

The month name is converted into a numeric representation using a CASE expression in the OLE DB Source query:

- January → 1
- February → 2
- March → 3
- ...
- December → 12

This ensures consistency in date representation.



3. Derived Column Transformation

A surrogate date_key is generated using a derived column expression:

(date_year * 10000) + (month_number * 100) + day_of_month

This creates a unique integer key in the format **YYYYMMDD**, which is used as the primary key of the DimDate table.

4. Data Load

The transformed data is loaded into the DimDate table using an OLE DB Destination component.

Validation(Data Loaded to SSMS):

SQLQuery3.s...PUTERS (54))* | Sathish-Diss... dbo.DimDate | Sathish-Diss... dbo.DimRoom | SQLQuery2.sq...PUTERS (63)) | Sathish-Dissa...- dbo.T

date_key	full_date	day_of_month	month_number	month_name	quarter	year	week_number	is_weekend
20150101	2015-01-01	1	1	January	1	2015	1	False
20150102	2015-01-02	2	1	January	1	2015	1	False
20150103	2015-01-03	3	1	January	1	2015	1	True
20150104	2015-01-04	4	1	January	1	2015	2	True
20150105	2015-01-05	5	1	January	1	2015	2	False
20150106	2015-01-06	6	1	January	1	2015	2	False
20150107	2015-01-07	7	1	January	1	2015	2	False
20150108	2015-01-08	8	1	January	1	2015	2	False
20150109	2015-01-09	9	1	January	1	2015	2	False
20150110	2015-01-10	10	1	January	1	2015	2	True
20150111	2015-01-11	11	1	January	1	2015	3	True
20150112	2015-01-12	12	1	January	1	2015	3	False
20150113	2015-01-13	13	1	January	1	2015	3	False
20150114	2015-01-14	14	1	January	1	2015	3	False
20150115	2015-01-15	15	1	January	1	2015	3	False
20150116	2015-01-16	16	1	January	1	2015	3	False
20150117	2015-01-17	17	1	January	1	2015	3	True
20150118	2015-01-18	18	1	January	1	2015	4	True
20150119	2015-01-19	19	1	January	1	2015	4	False
20150120	2015-01-20	20	1	January	1	2015	4	False
20150121	2015-01-21	21	1	January	1	2015	4	False
20150122	2015-01-22	22	1	January	1	2015	4	False
20150123	2015-01-23	23	1	January	1	2015	4	False
20150124	2015-01-24	24	1	January	1	2015	4	True
20150125	2015-01-25	25	1	January	1	2015	5	True
20150126	2015-01-26	26	1	January	1	2015	5	False
20150127	2015-01-27	27	1	January	1	2015	5	False
20150128	2015-01-28	28	1	January	1	2015	5	False
20150129	2015-01-29	29	1	January	1	2015	5	False

8 of 200

Error List

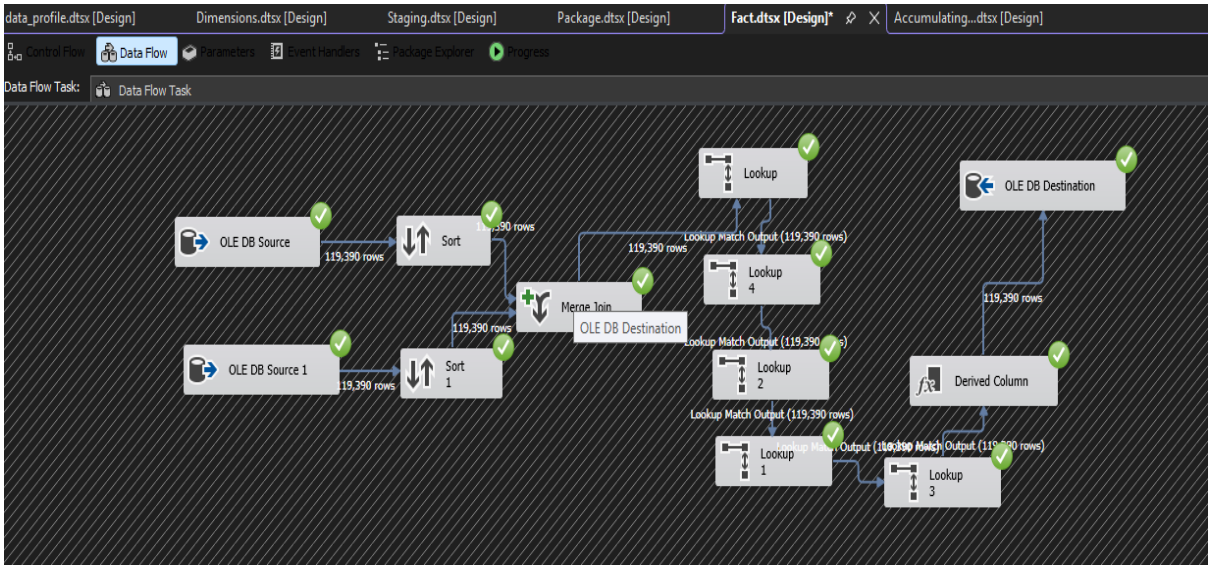
Entire Solution | 0 Errors | 0 Warnings | 0 Messages | Build + IntelliSense

Code | Description

5.5 Load FactBooking (Fact.dtsx)

Purpose:

The FactBooking table is designed to store transactional booking data. It integrates data from multiple sources, resolves surrogate keys from dimension tables, and loads final analytical records into the data warehouse.



Two OLE DB sources are used:

Source 1: stg_Booking

Contains core booking transactional data.

Source 2: stg_ReservationStatus

Contains reservation status updates.

Sorting Transformation:

Both data sources are sorted before joining:

- stg_Booking → sorted by **stg_booking_id (Ascending)**
- stg_ReservationStatus → sorted by **stg_reservation_id (Ascending)**

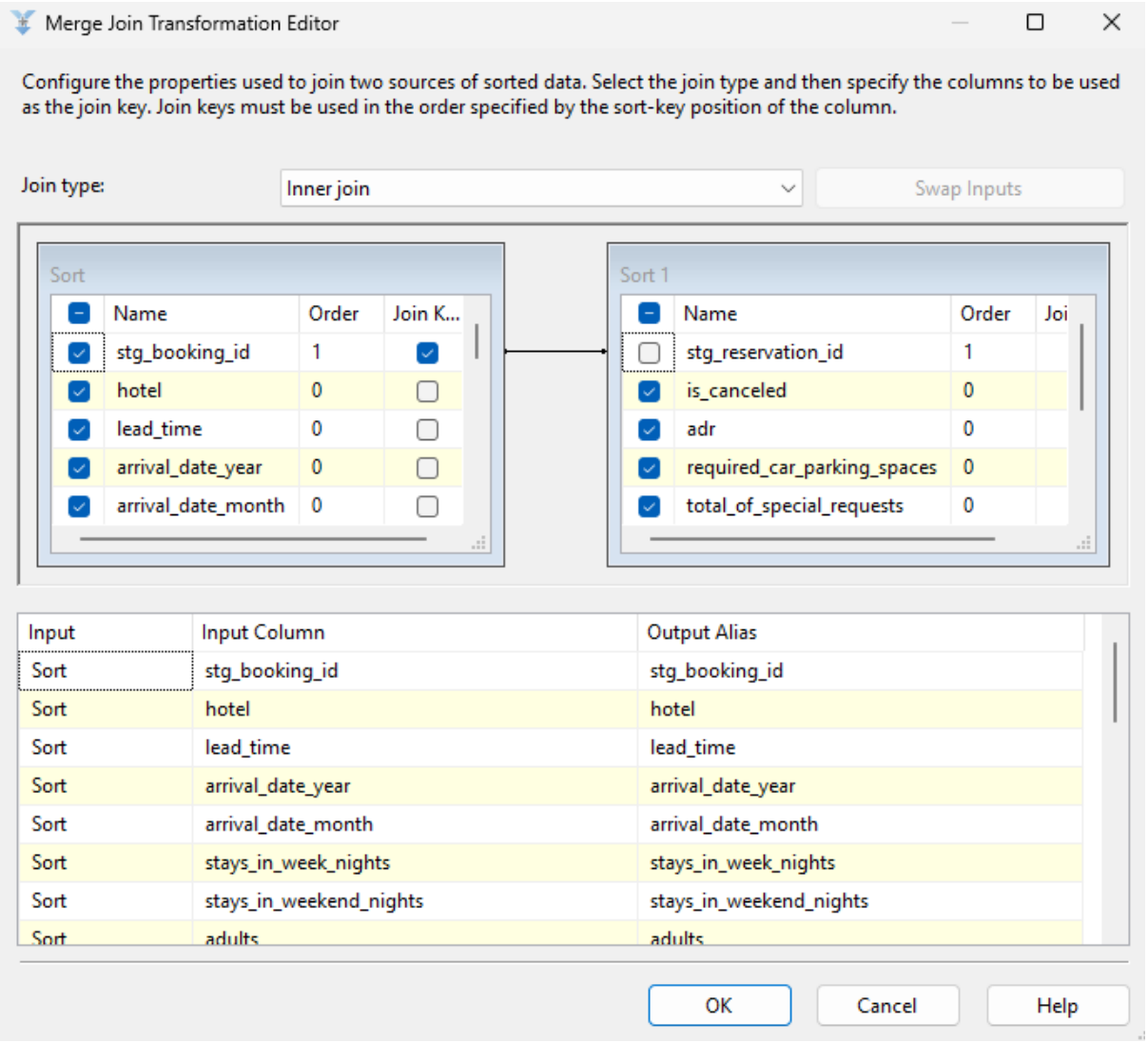
Sorting ensures proper alignment of records for merge operations.

Merge Join Transformation:

A Merge Join is used to combine both datasets into a unified transactional dataset.

Join Type: Based on matching booking/reservation identifiers

Purpose: To integrate booking details with reservation status information



Lookup Transformations (Surrogate Key Mapping) :

After merging, the data is passed through multiple Lookup transformations to replace natural keys with surrogate keys from dimension tables.

Lookups performed:

- DimHotel → hotel_key
- DimGuest → guest_key
- DimRoom → room_key
- DimAgent → agent_key
- DimDate → date_key

Purpose:

To maintain referential integrity in the data warehouse schema.

Derived Column Transformation (Accumulating Fact Logic) :

A Derived Column transformation is used to capture transaction creation time.

Column Added:

- accm_txn_create_time = GETDATE()

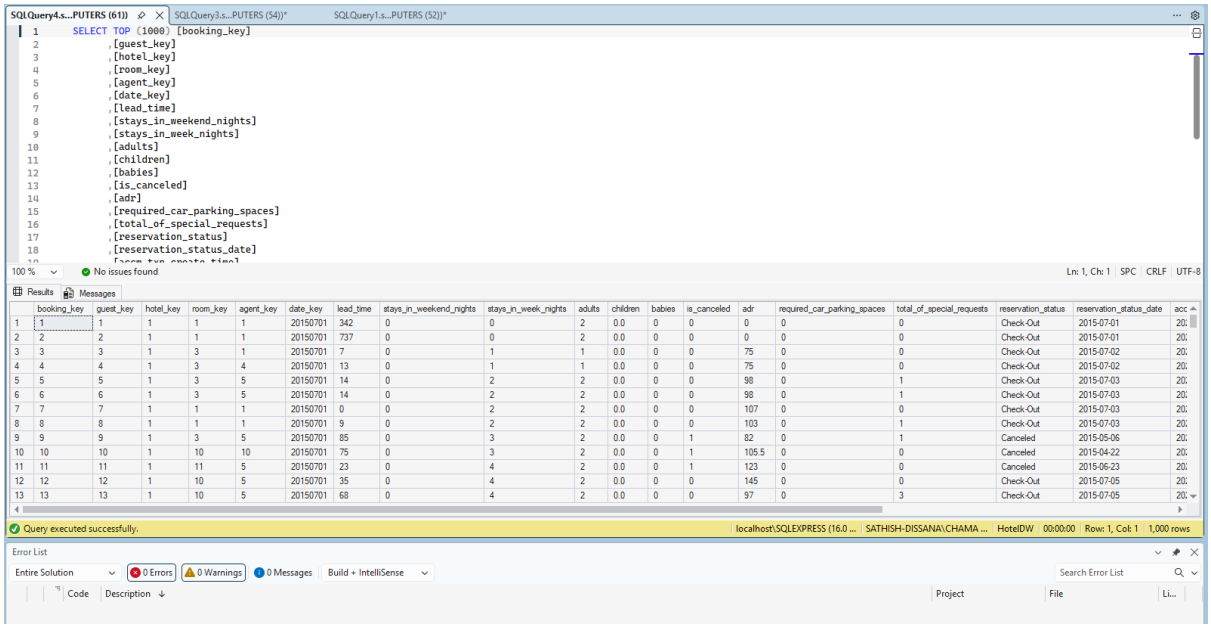
Purpose:

To record the exact time when the fact record is inserted into the warehouse.

OLE DB Destination :

The final transformed dataset is loaded into the FactBooking table in the data warehouse.

Validation(Data Loaded to SSMS):



SSIS Error Handling (OnError Event Handler):

An OnError event handler was implemented at the package level in the Fact.dtsx SSIS package. This captures runtime errors during ETL execution and logs them into an ETL_ErrorLog table along with the error message, timestamp, and package name. This provides basic monitoring and debugging support for the ETL process.

SQLQuery3.s...UTERS (54))* | Sathish-Diss...- dbo.DimDate | Sathish-Diss... dbo.DimRoom

```sql
USE HotelDW
GO

CREATE TABLE ETL_ErrorLog (
    ErrorMessage NVARCHAR(4000),
    ErrorTime DATETIME,
    PackageName NVARCHAR(100)
);
```

100 % | No issues found | Ln: 8, Ch: 3 | S

Messages

```
Commands completed successfully.

Completion time: 2026-04-19T06:09:35.9551768+05:30
```

Execute SQL Task Editor

Configure the properties required to run SQL statements and stored procedures using the selected connection.

- General
- Parameter Mapping
- Result Set
- Expressions

General

Enter SQL Query

```sql
INSERT INTO ETL_ErrorLog
(ErrorMessage, ErrorTime, PackageName)
VALUES (?, GETDATE(), ?)
```

OK | Cancel

Task

Task

SANA\SQLEXPRESS.HotelDW

ETL_ErrorLog(ErrorMessage, Erro

Browse... | Build Query... | Parse Query

OK | Cancel | Help

6. Task 6 - ETL Development - Accumulating fact tables

6.1 Accumulating Fact Table Overview

The accumulating fact table pattern was implemented to track the lifecycle of booking transactions from creation to completion. This approach enables analysis of processing duration and transaction status over time.

To support this, the **FactBooking** table was extended with the following additional attributes:

- **accm_txn_create_time** – stores the timestamp when the transaction record is initially created in the data warehouse
- **accm_txn_complete_time** – stores the timestamp when the transaction is completed (updated later)
- **txn_process_time_hours** – stores the total processing duration in hours between creation and completion

This design allows tracking of transaction progress and supports time-based performance analysis.

6.2 Step 1 - Extending the Fact Table

The FactBooking table schema was extended to include the accumulating fact columns.

- accm_txn_create_time → NOT NULL
- accm_txn_complete_time → NULL (initially unknown)
- txn_process_time_hours → NULL (calculated later)

These columns enable incremental updates to transaction records as new information becomes available.

```sql
USE HOTEL_DW
GO

ALTER TABLE FactBooking
ADD
    accm_txn_create_time DATETIME,
    accm_txn_complete_time DATETIME NULL,
    txn_process_time_hours AS
        DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time);
```

```
Messages
Commands completed successfully.

Completion time: 2026-03-30T11:01:03.9082026+05:30
```

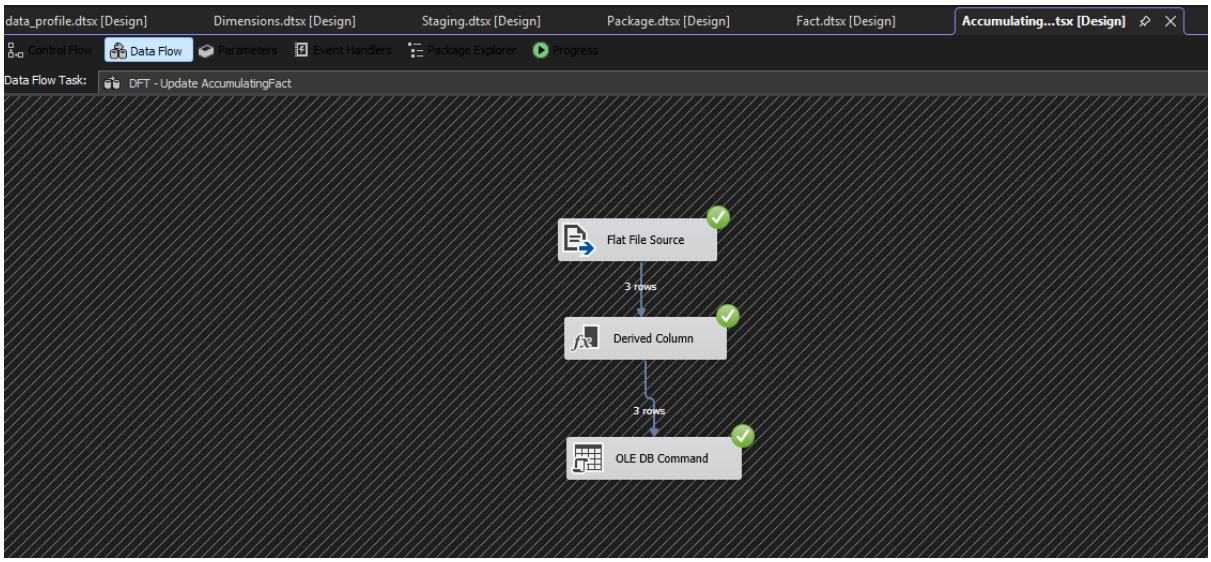
6.3 Step 2 - Setting accm_txn_create_time During Initial Load

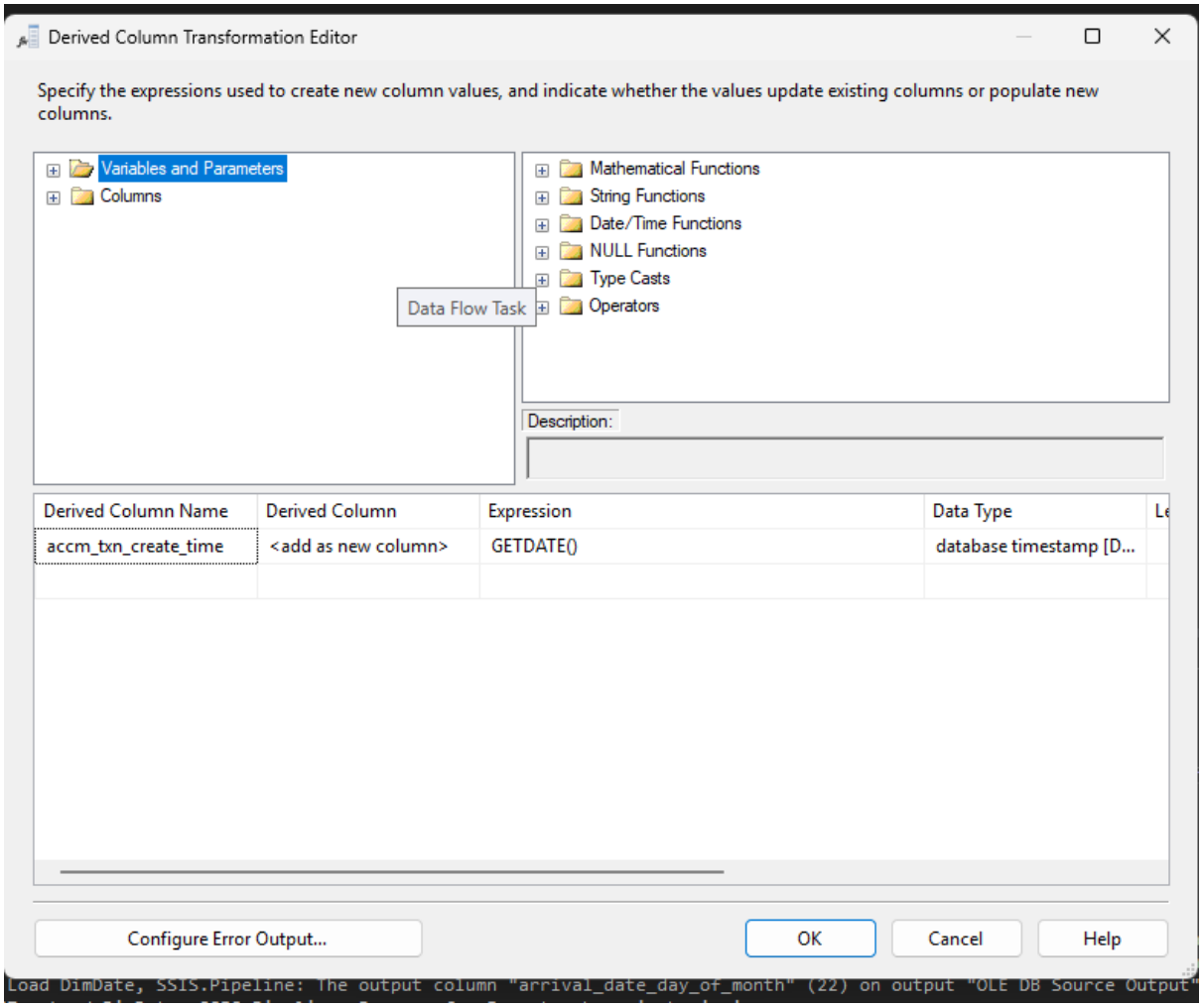
During the initial fact table load (Fact.dtsx), the accm_txn_create_time is populated using the current system date and time.

This is implemented using a **Derived Column transformation**:

GETDATE()

This ensures that every record inserted into the fact table contains a timestamp indicating when it was created in the data warehouse.




Derived Column Transformation Editor
Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.
Variables and Parameters
Columns
Data Flow Task
Mathematical Functions
String Functions
Date/Time Functions
NULL Functions
Type Casts
Operators
Description:
Derived Column Name
Derived Column
Expression
Data Type
accm_txn_create_time
<add as new column>
GETDATE()
database timestamp [D...
Configure Error Output...
OK
Cancel
Help
Load DimDate, SSIS.Pipeline: The output column "arrival_date_day_of_month" (22) on output "OLE DB Source Output"

6.4 Step 3 - Preparing the Completion Dataset

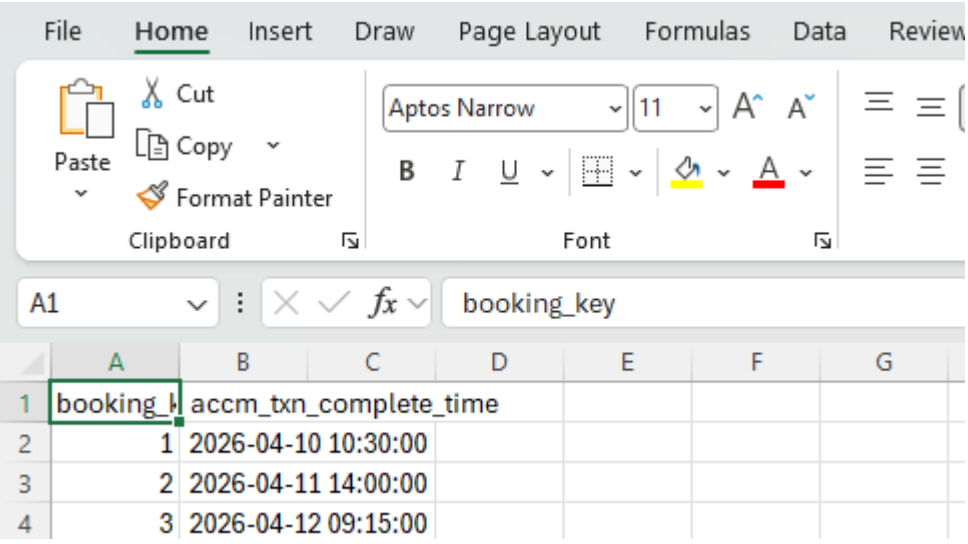
A separate dataset was created to simulate delayed transaction completion.

File: **txn_complete.csv**

Structure:

- booking_key – natural key to identify the transaction
- accm_txn_complete_time – timestamp of transaction completion

This dataset represents **late-arriving data**, where transaction completion occurs after the initial data load.



6.5 Step 4 - Update_AccumulatingFact.dtsx Package

A separate SSIS package named **Update_AccumulatingFact.dtsx** was developed to update the completion time and calculate processing duration.

Data Flow Design

Flat File Source :

Reads booking_key and accm_txn_complete_time from CSV

Derived Column (Optional Demonstration) :

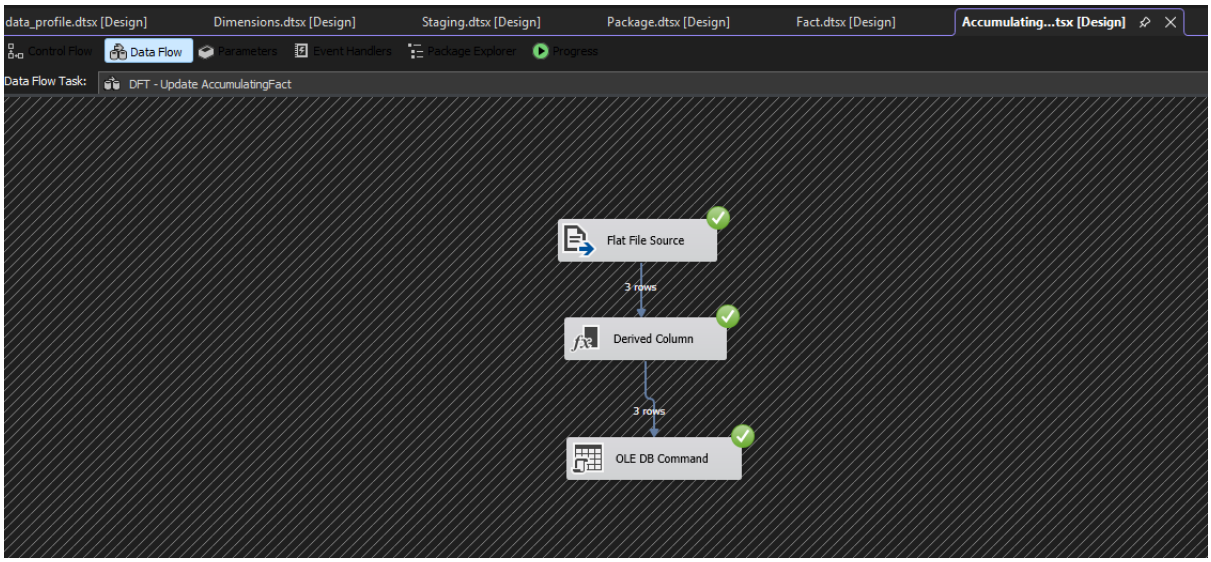
accm_txn_create_time = GETDATE()

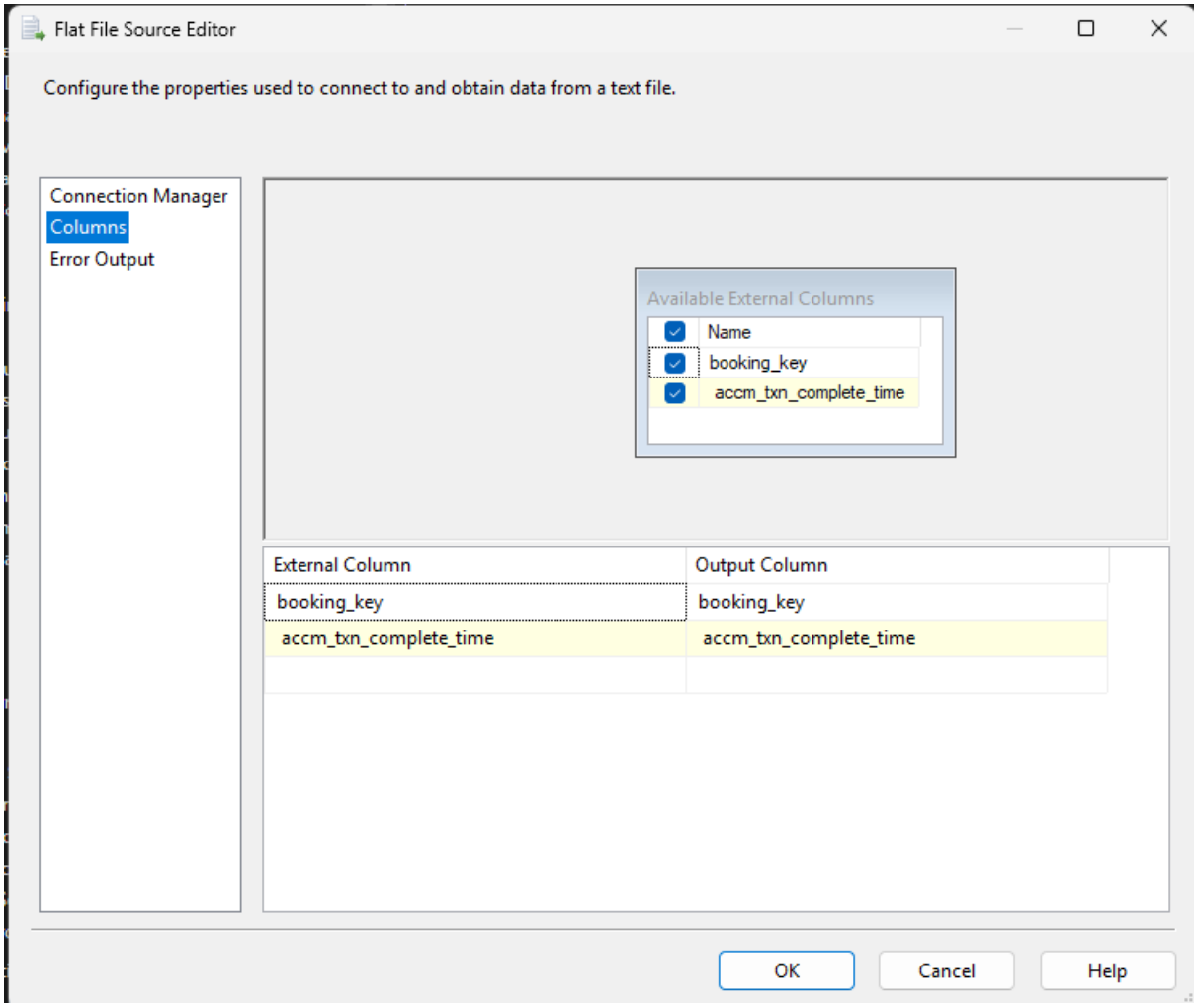
Included to demonstrate timestamp generation capability in SSIS

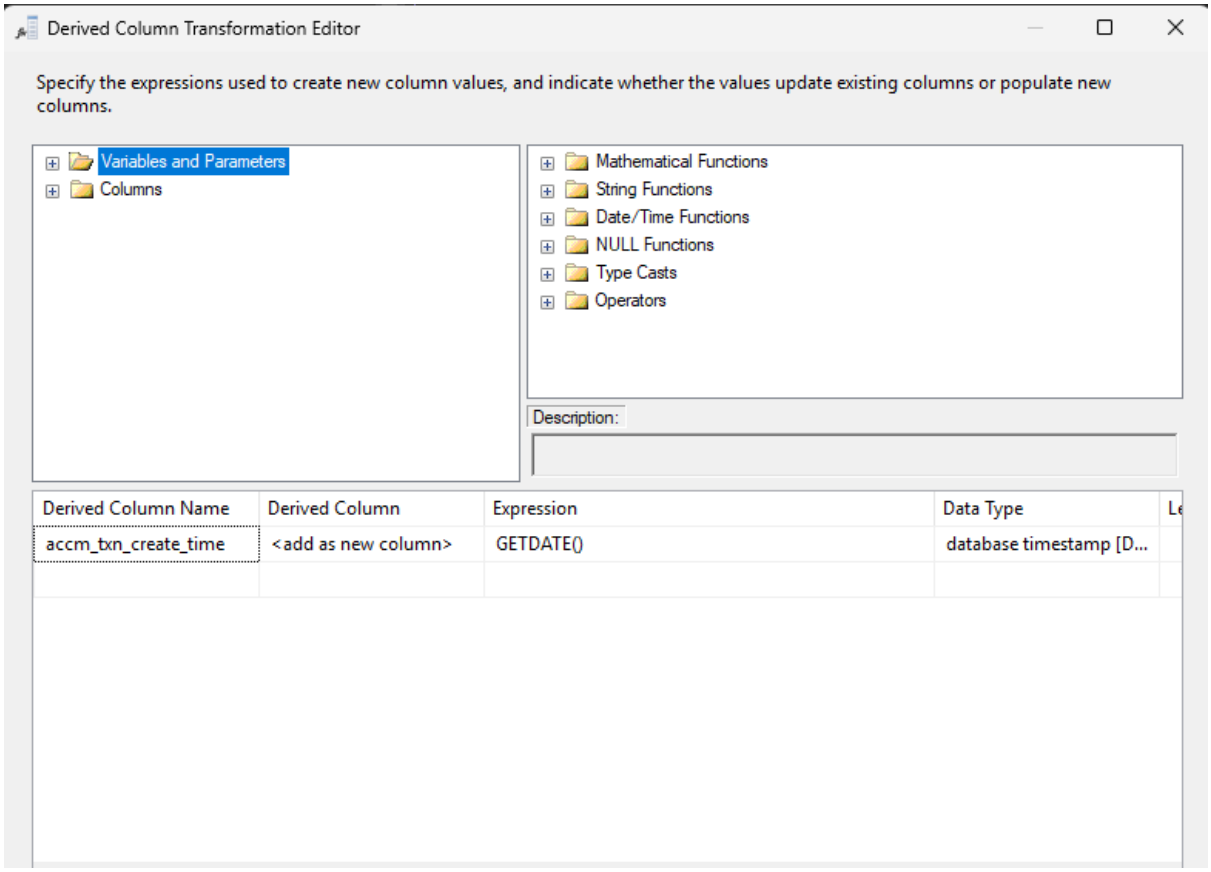
Not used in update logic (existing DB value is used instead)

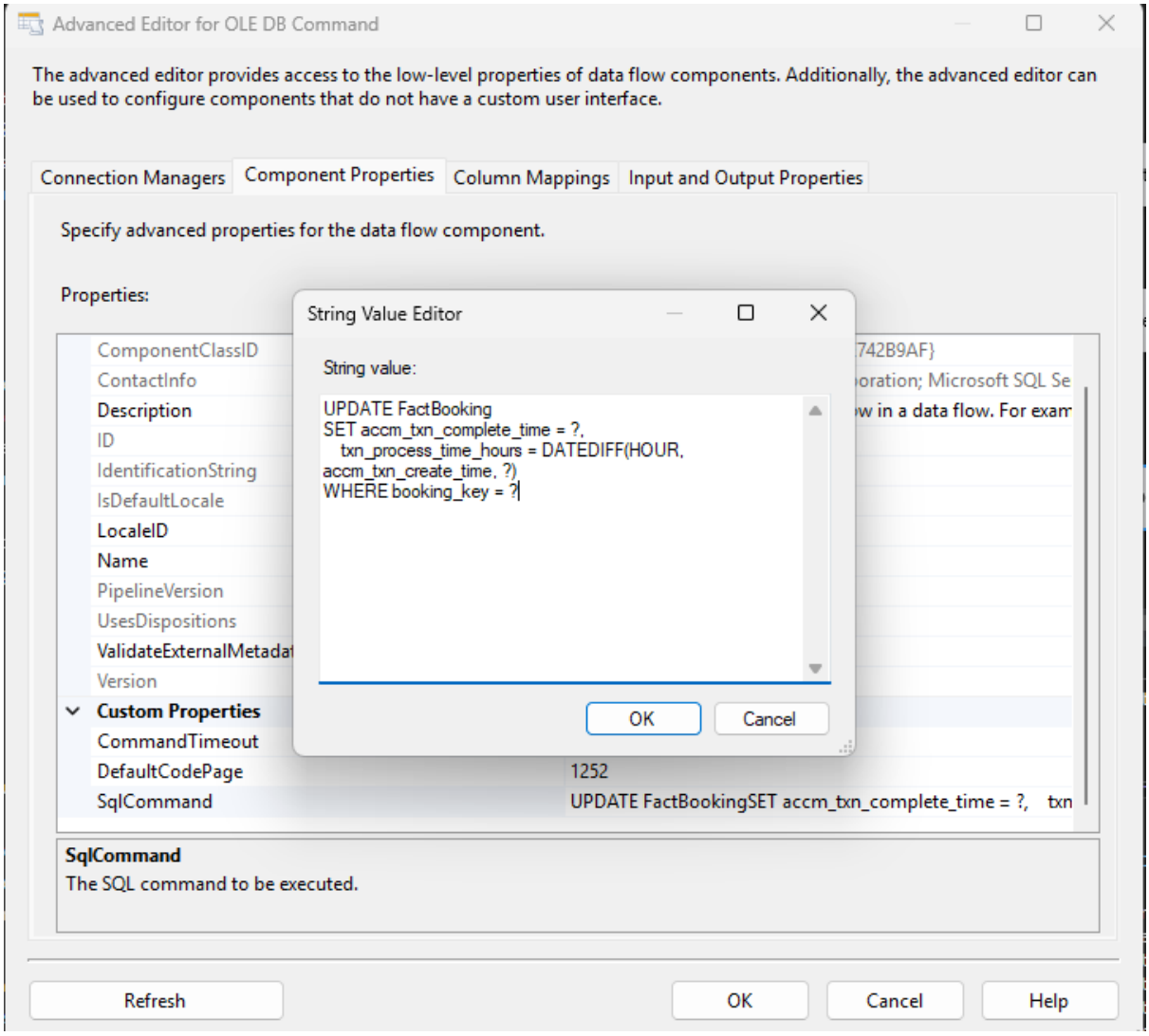
OLE DB Command :

Performs row-by-row updates on FactBooking


data_profile.dtsx [Design]
Dimensions.dtsx [Design]
Staging.dtsx [Design]
Package.dtsx [Design]
Fact.dtsx [Design]
Accumulating...tsx [Design]
Control Flow
Data Flow
Parameters
Event Handlers
Package Explorer
Progress
Data Flow Task:
DFT - Update AccumulatingFact
Flat File Source
3 rows
Derived Column
3 rows
OLE DB Command


Flat File Source Editor
Configure the properties used to connect to and obtain data from a text file.
Connection Manager
Columns
Error Output
Available External Columns
Name
booking_key
accm_txn_complete_time
External Column
Output Column
booking_key
booking_key
accm_txn_complete_time
accm_txn_complete_time
OK
Cancel
Help


Derived Column Transformation Editor
Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.
Variables and Parameters
Columns
Mathematical Functions
String Functions
Date/Time Functions
NULL Functions
Type Casts
Operators
Description:
Derived Column Name
Derived Column
Expression
Data Type
accm_txn_create_time
<add as new column>
GETDATE()
database timestamp [D...



SQL Update Logic

UPDATE FactBooking

SET accm_txn_complete_time = ?,

  txn_process_time_hours = DATEDIFF(HOUR, accm_txn_create_time, ?)

WHERE booking_key = ?

AND accm_txn_complete_time IS NULL

Processing Logic

- The accm_txn_complete_time is updated from the external dataset
- The txn_process_time_hours is calculated using SQL DATEDIFF
- The calculation uses the existing accm_txn_create_time stored in the fact table
- The condition accm_txn_complete_time IS NULL ensures that already completed transactions are not overwritten

6.6 Step 5 – Processing Time Calculation

The transaction processing duration is calculated dynamically during the update phase:

```
DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time)
```

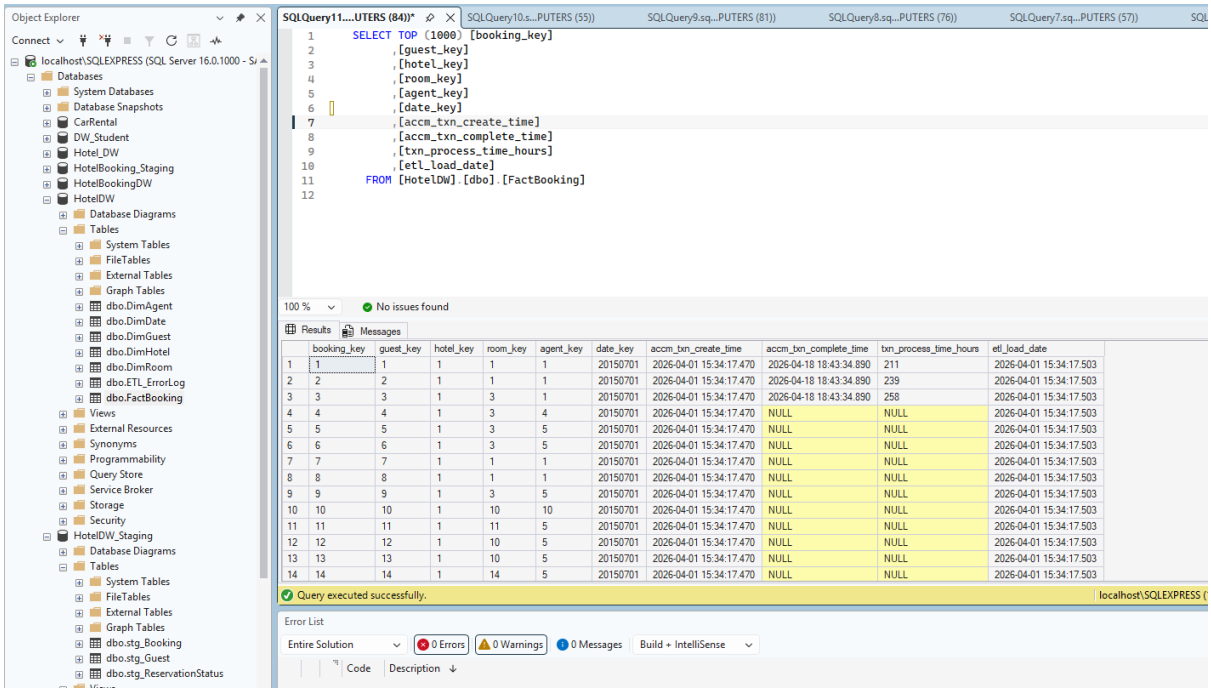
This ensures accurate measurement of the time taken for each transaction from creation to completion.

6.7 Validation

After executing the update package, validation queries were run in SQL Server Management Studio.

Validation checks included:

- Verifying that accm_txn_complete_time is updated correctly
- Ensuring txn_process_time_hours is calculated
- Confirming no duplicate or overwritten records



6.8 Summary

The accumulating fact table implementation successfully:

- Tracks transaction lifecycle from creation to completion

- Handles late-arriving data using a separate ETL process
- Calculates processing duration dynamically
- Prevents overwriting existing data
- Demonstrates advanced ETL techniques using SSIS